Bayesian-LoRA: Probabilistic Low-Rank Adaptation of Large Language Models

Moule Lin [1] **Shuhao Guan** [2] **Andrea Patane** [1] **David Gregg** [1] **Goetz Botterweck** [1]

Abstract

Large Language Models usually put more emphasis on accuracy and therefore, will guess even when not certain about the prediction, which is especially severe when fine-tuned on small datasets due to the inherent tendency toward miscalibration. In this work, we introduce Bayesian-LoRA, which reformulates the deterministic LoRA update as a probabilistic low-rank representation inspired by Sparse Gaussian Processes. We identify a structural isomorphism between LoRA's factorization and Kronecker-factored SGP posteriors, and show that LoRA emerges as a limiting case when posterior uncertainty collapses. We conduct extensive experiments on various LLM architectures across commonsense reasoning benchmarks. With only approximately 0.42M additional parameters and $\approx 1.2\times$ training cost relative to standard LoRA, Bayesian-LoRA significantly improves calibration across models up to 30B, achieving up to 84% ECE reduction and 76% NLL reduction while maintaining competitive accuracy for both in-distribution and out-of-distribution (OoD) evaluations.

1. Introduction

Fine-tuning large language models (LLMs) with parameter-efficient methods such as Low-Rank Adaptation (LoRA) (Hu et al., 2022) is now standard practice for adapting pre-trained models to downstream tasks (Ding et al., 2023; Xu et al., 2021; Malladi et al., 2023; Hu et al., 2024; Wang et al., 2024a; Shorinwa et al., 2025). By updating only a small set of low-rank matrices, LoRA achieves strong task accuracy with lower computational cost, memory footprint, and data requirements (Yin et al., 2024; Lin et al., 2024; Liu et al., 2025; Ye et al., 2024).

Although pre-trained models are reasonably well calibrated out of the box (Zhou et al., 2024; Wang, 2023), calibration often deteriorates after domain-specific fine-tuning, and as a result, models tend to become systematically overconfident (Liu et al., 2024; Mai et al., 2024; Zhou et al., 2023). Such degradation poses problems when LLMs are applied in safety-critical domains, such as autonomous driving (Tu et al., 2025; Wu et al., 2021), medical diagnosis (Savage et al., 2025), and others (Liu et al., 2025; Kim et al., 2025; Chuang et al., 2025). This motivates calibration-aware fine-tuning methods that can maintain or improve probability calibration during LLM training.

Limitations of existing approaches. Probabilistic methods can improve calibration through Bayesian neural networks (Yang et al., 2024), stochastic formulations (Lin et al., 2025a), and Sparse Gaussian Processes (Titsias, 2009; Burt et al., 2020). However, full Bayesian inference is computationally infeasible at LLM scale (Xue et al., 2021; Sankararaman et al., 2022), while efficient approximations like Laplace methods (Yang et al., 2024) apply calibration corrections *after* training. BLoB (Wang et al., 2024b) trains Bayesian LoRA weights via backpropagation but requires mean-field assumptions over the full LoRA parameter space.

We propose **Bayesian-LoRA**, a calibration-aware fine-tuning framework derived from a structural observation: the Kronecker-factored conditional distribution in Sparse Gaussian Process (SGP) inference produces weight updates $M_W(U) = T_r\, U\, T_c$, which exhibits a *structural isomorphism* (in the functional sense of shared bilinear form, not strict algebraic equivalence) with LoRA's factorization $\Delta W = BA$. The projectors T_r, T_c play analogous roles to LoRA's B, A matrices, and the inducing variable U replaces the deterministic core with a stochastic one. This suggests that *LoRA can be naturally embedded within a probabilistic framework*, where deterministic LoRA emerges as a limiting case. By maintaining a non-degenerate variational posterior over U, enriched by a normalizing flow (Lin et al., 2025b), we obtain a probabilistic generalization with calibrated uncertainty. This design offers three advantages: (i) uncertainty is modeled in a low-rank inducing space, keeping overhead minimal; (ii) a closed-form KL term avoids expensive Hessian computations; and (iii) calibration is optimized end-to-end during training.

We evaluate Bayesian-LoRA on six commonsense reason-

[1]School of Computer Science and Statistics, Trinity College Dublin, Dublin, Ireland [2]School of Computer Science, University College Dublin, Dublin, Ireland. Correspondence to: Goetz Botterweck <goetz.botterweck@tcd.ie>.

Preprint. April 16, 2026.

ing benchmarks with LLaMA 2-7B, generative language modeling on WikiText-2, and mathematical reasoning with Qwen2.5-14B and Qwen3-30B-A3B. The contributions are:

- **Structural Isomorphism.** We identify a structural isomorphism (shared bilinear functional form, not strict algebraic equivalence) between Kronecker-factored SGP posteriors and LoRA's factorization. Under limiting conditions (point-mass posterior and vanishing conditional noise), Bayesian-LoRA reduces to a deterministic bilinear update, motivating our probabilistic generalization.
- **Flow-Augmented Variational Inference.** We enhance the posterior using normalizing flows and derive a closed-form ELBO that is independent of weight dimensionality. This enables calibration-aware training with minimal overhead ($\approx 1.2\times$ training time, ≈ 0.42M additional parameters).
- **Evaluation.** Tested across commonsense reasoning, text generation, and math tasks (including 14B dense and 30B MoE models), Bayesian-LoRA improves NLL (Negative Log-Likelihood) and achieves strong calibration under distribution shift while maintaining competitive accuracy.

2. Related Work

Model calibration matters for trustworthy machine learning. Although modern neural networks can achieve high predictive accuracy, they are often poorly calibrated and produce overconfident predictions (Guo et al., 2017).

Post-hoc methods improve calibration by scaling model predictions after training (Kull et al., 2019; Guo et al., 2017). However, they cannot prevent weight-level uncertainty degradation, and under distribution shift, post-hoc calibration often becomes less effective (Desai & Durrett, 2020).

Bayesian methods quantify uncertainty by BNNs (Izmailov et al., 2021; Lin et al., 2025a; Kweon et al., 2025), Laplace approximations (Yang et al., 2024), and Gaussian processes (Ranković & Schwaller, 2025), but incur high cost at LLM scale. Full-model methods (including BLoB (Wang et al., 2024b)) require approximate inference over entire parameter spaces (Graves, 2011; Blundell et al., 2015); Laplace methods depend on Hessians whose cost grows with model size (Daxberger et al., 2021; Ritter et al., 2018); GPs scale cubically in data (Seeger, 2004; Quinonero-Candela & Rasmussen, 2005; Ritter et al., 2021); and ensembles multiply inference cost (Lakshminarayanan et al., 2017). FFG-U (Ritter et al., 2021) proposed inducing weights by Kronecker-factorized decomposition of SGP weights that introduces a low-dimensional inducing weight matrix U. We therefore inspired model uncertainty within the low-rank subspace of LoRA, combining sparse GP with normalizing flows to induce ΔW with low overhead.

Parameter-Efficient Fine-Tuning (PEFT) introduces a small set of additional parameters while keeping the base LLM frozen, reducing fine-tuning cost and memory footprint. Methods include adapter-based tuning (Houlsby et al., 2019; Pfeiffer et al., 2021), prompt/prefix tuning (Lester et al., 2021; Li & Liang, 2021), and low-rank adaptation (LoRA and its variants) (Hu et al., 2022; Zhang et al., 2023; Dettmers et al., 2023). However, PEFT techniques employ deterministic updates and do not model uncertainty or optimize calibration objectives within the adapter subspace.

We integrate Sparse Gaussian Processes with LoRA by exploiting a structural isomorphism (shared bilinear form) between Kronecker-factored SGP posteriors and LoRA's factorization, bringing probabilistic inference to LLMs while maintaining PEFT-level efficiency.

3. Preliminaries

This section reviews LoRA (§3.1) and introduces the variational sparse inducing weight model (§3.2).

3.1. LoRA: Low-Rank Adaptation

Low-Rank Adaptation (LoRA) (Hu et al., 2022) parameterizes the weight update ΔW as a low-rank factorization $\frac{\alpha}{r} BA$ with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$, where α is a fixed scaling factor. The pre-trained weight $W_{\text{pre}} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ remains frozen during training; only A and B are optimized:

$$\Delta W = \frac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d_{\text{out}} \times r},\ A \in \mathbb{R}^{r \times d_{\text{in}}} \tag{1}$$

$$y = \big(W_{\text{pre}} + \Delta W\big)x = W_{\text{pre}}\, x + \frac{\alpha}{r} B\big(Ax\big) \tag{2}$$

where $x \in \mathbb{R}^{d_{\text{in}}}$ is the layer input and y the output. This reduces the number of trainable parameters from $d_{\text{in}} d_{\text{out}}$ (full fine-tuning) to $r(d_{\text{in}} + d_{\text{out}})$, and the adapter can be merged into W_{pre} at inference time with no added latency. LoRA is typically applied to Transformer projection layers (e.g., W_q, W_k, W_v, W_o). However, the update ΔW is *deterministic*: it provides a single point estimate of the weight perturbation, capturing no information about uncertainty.

3.2. Variational Sparse Inducing Weight Model

Core idea. Rather than placing a distribution directly on $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, we introduce a compact *inducing matrix* $U \in \mathbb{R}^{r \times c}$ with $r \ll d_{\text{out}}$, $c \ll d_{\text{in}}$ that controls the distribution over W (Ritter et al., 2021; Yang et al., 2024; Snelson & Ghahramani, 2005). As in Sparse GPs, the low-dimensional U acts as a sufficient statistic for the posterior over W, keeping inference tractable.

Prior and variational posterior on U. We place a Gaussian (matrix-normal) prior on U:

$$p(U) = \mathcal{N}\big(\mathrm{vec}(U) \mid \mathbf{0},\, K_U\big),\; K_U = K_c \otimes K_r \quad (3)$$

where $K_r \in \mathbb{R}^{r\times r}$ and $K_c \in \mathbb{R}^{c\times c}$ are learnable row and column covariance factors, and $\otimes$ denotes the Kronecker product. The variational posterior is:

$$q(U) = \mathcal{N}\big(\mathrm{vec}(U) \mid \mathbf{m}, \mathbf{S}\big) \quad (4)$$

where $\mathbf{m}$ and $\mathbf{S}$ are the variational mean and covariance. Whitening can be applied so that the prior on U becomes standard normal, simplifying the KL computation.

Conditional distribution of W given U. Given the inducing variables U, the weight matrix W follows a Gaussian conditional (Lin et al., 2025a):

$$p(W \mid U) = \mathcal{N}\big(W \mid M_W(U),\; \lambda^2 \Sigma_W\big) \quad (5)$$

where $\lambda > 0$ is a learnable scale parameter governing the conditional variance. The covariance factors are parameterized as:

$$K_r = Z_r Z_r^\top + D_r^2, \qquad K_c = Z_c Z_c^\top + D_c^2 \quad (6)$$

with $Z_r \in \mathbb{R}^{r\times r}$, $Z_c \in \mathbb{R}^{c\times c}$ learnable, and D_r, D_c diagonal noise matrices. The projection operators

$$T_r = Z_r^\top K_r^{-1}, \qquad T_c = K_c^{-1} Z_c \quad (7)$$

define the conditional mean of W as a bilinear projection (derivation in Appendix A):

$$M_W(U) = T_r\, U\, T_c \quad (8)$$

Marginal distribution and ELBO. The marginal distribution over W is obtained by integrating out U:

$$q(W) = \int p(W \mid U)\, q(U)\, \mathrm{d}U \quad (9)$$

Optimizing only the low-dimensional variational parameters of U thus suffices to approximate the high-dimensional posterior over W. The variational evidence lower bound takes the form:

$$\mathcal{L} = \mathbb{E}_{q(W)}\big[\log p(\mathcal{D} \mid W)\big] - \mathrm{KL}\big(q(U) \,\|\, p(U)\big) - \mathbb{E}_{q(U)}\Big[\mathrm{KL}\big(q(W \mid U) \,\|\, p(W \mid U)\big)\Big] \quad (10)$$

where the three terms are: expected log-likelihood, KL over inducing variables, and conditional KL. Since $q(W \mid U)$ and $p(W \mid U)$ share the same mean and differ only by λ, the conditional KL admits a *closed-form* expression independent of U (see §4).

Proposition 3.1 (KL invariance under T_ϕ)**.** *Let T_ϕ be invertible and set $\Delta W = T_\phi(U)$. With pushforwards $q_\phi := T_{\phi\#} q_\psi$ and $p_\phi := T_{\phi\#} p$, we have*

$$\mathrm{KL}\big(q_\phi \,\|\, p_\phi\big) = \mathrm{KL}\big(q_\psi \,\|\, p\big). \quad (11)$$

Hence, the ELBO written in U-space equals the ELBO written in ΔW-space. Proof. *See Appendix A.1.*

This result guarantees that we can optimize the ELBO in the compact U-space while the induced posterior over ΔW remains consistent.

4. Methodology: Bayesian-LoRA

Bayesian-LoRA replaces the deterministic low-rank update of LoRA with a probabilistic formulation. The LoRA weight matrices A and B are treated as random variables, each modeled by a per-layer inducing-variable posterior enriched by a normalizing flow. We describe how LoRA maps to this Bayesian formulation (§4.1), then derive the training objective (§4.2).

4.1. From LoRA to Bayesian-LoRA

Probabilistic low-rank update. Recall that LoRA uses a deterministic rank-r update:

$$\Delta W_{\text{LoRA}} = \tfrac{\alpha}{r} BA, \qquad B \in \mathbb{R}^{d_{\text{out}}\times r},\; A \in \mathbb{R}^{r\times d_{\text{in}}} \quad (12)$$

In Bayesian-LoRA, we replace this with a stochastic update. For each target layer, we introduce a low-dimensional inducing matrix $U \in \mathbb{R}^{r\times c}$ and "diffuse" its information into the high-dimensional weight space via the conditional Gaussian $p(W \mid U)$ (Eq. (5)). The conditional mean $M_W(U) = T_r\, U\, T_c$ (Eq. (8)) acts as a stochastic analog of the LoRA matrices A and B: each Monte Carlo sample of U produces a different weight realization, and the spread of these realizations encodes epistemic uncertainty.

Compared to LoRA's deterministic $\Delta W = \frac{\alpha}{r} BA$, Bayesian-LoRA randomizes this update and quantifies uncertainty: ΔW is induced by the posterior uncertainty of U.

Normalizing flow for posterior flexibility. A purely Gaussian posterior on U may be too restrictive for capturing weight-space uncertainty of LLMs. Following Lin et al. (2025a), we enrich the variational family while keeping the parameter count small by placing a normalizing flow (Rezende & Mohamed, 2015; Papamakarios et al., 2017) on top of a diagonal-Gaussian base distribution:

$$\begin{aligned} q_0(U_0) &= \mathcal{N}\big(\mathrm{vec}(U_0) \mid \mathbf{m},\, \mathrm{diag}(\boldsymbol{\sigma}^2)\big),\; \boldsymbol{\sigma} \in \mathbb{R}^{rc}_{>0}, \\ U &= T_\phi(U_0) \end{aligned} \quad (13)$$

Here, T_ϕ is an invertible, differentiable map; in practice, we use a lightweight row-wise Masked Autoregressive Flow

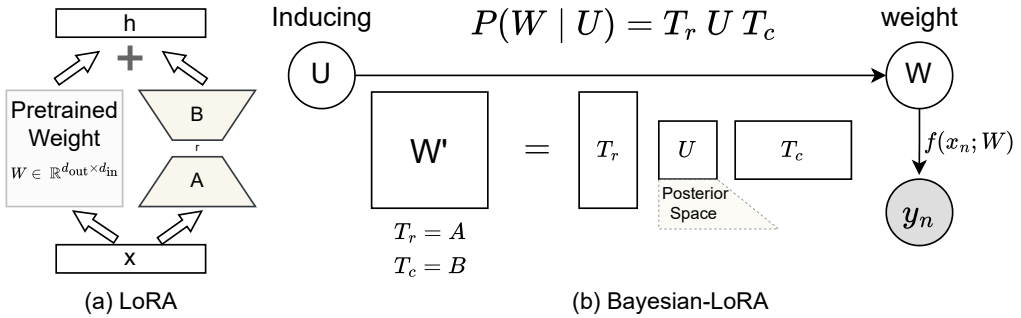

Figure 1. **Bayesian-LoRA** overview. Inducing variables U are transformed by a flow T_ϕ and projected via conditional Gaussians to produce stochastic LoRA matrices A, B. The effective weight $W_{\text{eff}} = W_{\text{pre}} + \frac{\alpha}{r} BA$ is used for the forward pass; Monte Carlo samples capture epistemic uncertainty.

(MAF). When T_ϕ is the identity, $q_\phi(U)$ reduces to the diagonal-Gaussian baseline; increasing flow capacity improves posterior expressiveness and downstream calibration. The Gaussian prior on U follows Eq. (3): $p(U) = \mathcal{N}\big(\text{vec}(U) \mid \mathbf{0}, K_c \otimes K_r\big)$.

4.2. Training Objective: Flow-Augmented ELBO

By the change-of-variables formula, the flow-transformed density of U is:

$$\log q_\phi(U) = \log q_0\big(T_\phi^{-1}(U)\big) - \log\Big|\det J_{T_\phi}\big(T_\phi^{-1}(U)\big)\Big| \quad (14)$$

Substituting $q_\phi(U)$ into the standard SVGP ELBO (Eq. (10)) yields:

$$\begin{aligned}\mathcal{L}_{\text{ELBO}} &= \mathbb{E}_{U_0 \sim q_0,\, \epsilon}\big[\log p(\mathcal{D} \mid W)\big] \\ &- \mathbb{E}_{U_0 \sim q_0}\Big[\log q_0(U_0) - \log\big|\det J_{T_\phi}(U_0)\big| \\ &- \log p\big(T_\phi(U_0)\big)\Big] - \frac{D}{2}\Big(\lambda^2 - 1 - 2\log\lambda\Big)\end{aligned} \quad (15)$$

where $d_W^{(\ell)} = d_{\text{out}}^{(\ell)} d_{\text{in}}^{(\ell)}$ is the number of weight entries in the ℓ-th replaced layer and $D = \sum_\ell d_W^{(\ell)}$ is the total across all such layers. The three terms correspond to: (1) the expected log-likelihood, approximated via Monte Carlo sampling; (2) the KL divergence over inducing variables $\text{KL}(q_\phi(U) \,\|\, p(U))$, which by Proposition 3.1 equals the KL in ΔW-space; and (3) the conditional KL $\text{KL}(q(W \mid U) \,\|\, p(W \mid U))$, which has a closed form that is independent of U. See Appendix A.6 for detailed interpretations.

The Jacobian determinant $\log|\det J_{T_\phi}(U_0)|$ is computed efficiently by the autoregressive structure of the MAF.

Structural isomorphism with LoRA. The Kronecker decomposition $K_U = K_c \otimes K_r$ yields the conditional mean $M_W(U) = T_r\, U\, T_c$ (Eq. (8)), which exhibits a *structural isomorphism* with LoRA's $\Delta W = \frac{\alpha}{r} BA$ in the sense of shared bilinear functional form: the left projector $T_r \in \mathbb{R}^{d_{\text{out}} \times r}$ plays an analogous role to B, the right projector $T_c \in \mathbb{R}^{c \times d_{\text{in}}}$ to A, and the inducing variable $U \in \mathbb{R}^{r \times c}$ replaces the deterministic product with a stochastic low-rank core. When the inducing dimensions match the LoRA rank ($r = c$), Bayesian-LoRA produces weight updates in the same low-rank subspace while adding uncertainty quantification. We emphasize that this is a *functional* isomorphism (both produce low-rank bilinear weight updates), not a claim of strict algebraic equivalence; standard LoRA directly optimizes (A, B) as free parameters, whereas Bayesian-LoRA optimizes U with projection operators T_r, T_c derived from the covariance structure.

Corollary 4.1 (Deterministic limit)**.** *Let $q(U) = \delta(U - U^*)$ be a point-mass posterior and $\lambda \to 0$. Then the Bayesian-LoRA weight update reduces to the deterministic form $\Delta W = T_r\, U^*\, T_c$, recovering a bilinear low-rank update.*

Proof. Under $q(U) = \delta(U - U^*)$, sampling $U \sim q$ yields U^* with probability one. The conditional mean (Eq. (8)) gives $M_W(U^*) = T_r\, U^*\, T_c$. As $\lambda \to 0$, the conditional noise $\lambda\, \Sigma_W^{1/2} \varepsilon \to 0$ (Eq. (5)), so $W = M_W(U^*)$ deterministically. □

As illustrated in Figure 1, each Monte Carlo sample of U produces a different weight realization through this bilinear structure, and the spread of realizations encodes epistemic uncertainty. The complete training procedure is given in Algorithm 1.

5. Experiments

We empirically evaluate Bayesian-LoRA along three dimensions: (1) accuracy under a compute budget comparable to standard LoRA; (2) calibration and likelihood improvements; and (3) cost-effectiveness relative to existing Bayesian and post-hoc methods.

We use LLaMA 2-7B, Qwen2.5-14B-Instruct, and Qwen3-30B-A3B as base backbones, applying LoRA adapters to the Query (Q), Key (K), and LM Head weight matrices via the PEFT library (Mangrulkar et al., 2022). The corresponding

Table 1. Results on six commonsense reasoning benchmarks. We report Accuracy (ACC ↑), Expected Calibration Error (ECE ↓; 15 bins), and Negative Log-Likelihood (NLL ↓) at the early-stop checkpoint of each method. Values are mean±std over three seeds. Inducing-point dimensions are set to $r_{\text{ind}} = c_{\text{ind}} = 9$ (matching the LoRA rank) for a fair parameter-count comparison.

Metrics	Methods	WG-S	ARC-C	ARC-E	WG-M	OBQA	BoolQ
ACC ↑	MAP (Hu et al., 2022)	68.00 ± 0.21	64.90 ± 1.10	85.20 ± 0.60	73.70 ± 0.90	77.70 ± 0.80	85.80 ± 0.40
	Dropout (Gal & Ghahramani, 2016)	66.70 ± 0.30	64.90 ± 1.90	85.10 ± 0.50	73.50 ± 0.90	77.70 ± 0.20	85.90 ± 0.40
	Ckpt Ens (Huang et al., 2017)	66.70 ± 0.30	64.90 ± 1.10	85.20 ± 0.60	73.80 ± 1.00	78.20 ± 0.20	85.40 ± 0.30
	Temp (Guo et al., 2017)	67.00 ± 0.60	64.90 ± 1.10	85.20 ± 0.60	73.70 ± 0.90	77.70 ± 0.80	85.80 ± 0.40
	BBB (Blundell et al., 2015)	56.54 ± 7.87	68.13 ± 1.27	77.0 ± 0.2	73.63 ± 2.44	77.02 ± 0.2	83.17 ± 0.53
	LLLA (post-hoc) (Yang et al., 2024)	66.90 ± 0.50	66.10 ± 0.60	84.80 ± 0.50	73.70 ± 0.90	77.60 ± 0.70	85.80 ± 0.40
	LA (post-hoc) (Yang et al., 2024)	66.90 ± 0.60	66.90 ± 1.10	85.40 ± 0.40	73.70 ± 1.00	78.10 ± 0.70	85.80 ± 0.40
	BLoB (N=10) (Wang et al., 2024b)	69.07 ± 0.34	68.81 ± 1.09	85.56 ± 0.35	73.69 ± 0.17	81.52 ± 0.74	$\mathbf{86.99 \pm 0.24}$
	Bayesian-LoRA (S = 4) (End-to-End)	$\mathbf{70.90 \pm 0.1}$	$\mathbf{68.90 \pm 0.2}$	$\mathbf{85.91 \pm 0.3}$	$\mathbf{74.30 \pm 0.2}$	$\mathbf{81.60 \pm 0.1}$	86.10 ± 0.2
ECE ↓	MAP (Hu et al., 2022)	30.80 ± 1.80	26.10 ± 1.40	8.90 ± 0.30	24.90 ± 1.30	9.80 ± 1.00	7.40 ± 0.10
	Dropout (Gal & Ghahramani, 2016)	29.50 ± 1.60	25.60 ± 0.70	8.80 ± 0.60	23.50 ± 1.20	8.80 ± 0.80	7.50 ± 0.10
	Ckpt Ens (Huang et al., 2017)	25.20 ± 1.60	26.10 ± 1.40	8.90 ± 0.30	22.80 ± 1.40	4.70 ± 0.50	3.20 ± 0.50
	Temp (Guo et al., 2017)	12.80 ± 0.90	$\mathbf{4.60 \pm 1.00}$	4.70 ± 0.80	6.30 ± 1.60	7.20 ± 2.60	2.50 ± 0.30
	BBB (Blundell et al., 2015)	21.81 ± 12.95	26.23 ± 1.47	12.28 ± 0.58	15.76 ± 4.71	11.38 ± 1.07	3.74 ± 0.10
	LLLA (post-hoc) (Yang et al., 2024)	11.60 ± 1.30	5.60 ± 2.10	4.20 ± 0.30	3.80 ± 1.40	5.40 ± 0.40	1.70 ± 0.50
	LA (post-hoc) (Yang et al., 2024)	7.80 ± 1.90	7.50 ± 1.20	$\mathbf{3.40 \pm 0.80}$	4.80 ± 1.60	$\mathbf{3.50 \pm 0.40}$	1.90 ± 0.30
	BLoB (N=10) (Wang et al., 2024b)	9.35 ± 1.37	9.59 ± 1.88	3.64 ± 0.53	3.01 ± 0.12	3.77 ± 1.47	$\mathbf{1.41 \pm 0.19}$
	Bayesian-LoRA (S = 4) (End-to-End)	$\mathbf{4.90 \pm 0.20}$	9.20 ± 0.70	5.30 ± 0.10	$\mathbf{3.00 \pm 0.10}$	5.70 ± 0.20	2.10 ± 0.60
NLL ↓	MAP (Hu et al., 2022)	2.75 ± 0.57	1.64 ± 0.19	0.54 ± 0.03	2.43 ± 0.50	0.71 ± 0.03	0.43 ± 0.01
	Dropout (Gal & Ghahramani, 2016)	2.54 ± 0.49	1.55 ± 0.16	0.52 ± 0.04	2.12 ± 0.35	0.71 ± 0.04	0.43 ± 0.01
	Ckpt Ens (Huang et al., 2017)	1.31 ± 0.04	1.64 ± 0.18	0.54 ± 0.03	1.89 ± 0.24	0.65 ± 0.02	0.35 ± 0.01
	Temp (Guo et al., 2017)	0.68 ± 0.01	0.90 ± 0.01	0.43 ± 0.02	0.58 ± 0.01	0.67 ± 0.02	0.35 ± 0.00
	BBB (Blundell et al., 2015)	1.40 ± 0.55	2.23 ± 0.04	0.91 ± 0.06	0.84 ± 0.15	0.66 ± 0.05	0.31 ± 0.00
	LLLA (post-hoc) (Yang et al., 2024)	0.68 ± 0.01	0.94 ± 0.02	0.44 ± 0.01	0.56 ± 0.01	0.66 ± 0.02	0.35 ± 0.00
	LA (post-hoc) (Yang et al., 2024)	0.66 ± 0.02	0.86 ± 0.02	0.41 ± 0.02	0.55 ± 0.01	0.62 ± 0.01	0.34 ± 0.00
	BLoB (N=10) (Wang et al., 2024b)	$\mathbf{0.63 \pm 0.01}$	0.78 ± 0.02	0.40 ± 0.01	$\mathbf{0.54 \pm 0.00}$	0.50 ± 0.01	0.31 ± 0.00
	Bayesian-LoRA (S = 4) (End-to-End)	0.79 ± 0.01	$\mathbf{0.77 \pm 0.05}$	$\mathbf{0.38 \pm 0.09}$	0.62 ± 0.11	$\mathbf{0.49 \pm 0.02}$	$\mathbf{0.29 \pm 0.01}$

Table 2. Language modeling on **WikiText-2** (validation/test). We report NLL ↓, Brier score ↓, and ECE ↓ (15 bins) for both *all tokens* and the *top 5% most uncertain tokens* (selected by MAP predictive entropy).

	All tokens						Top 5% entropy tokens					
	Validation			Test			Validation			Test		
Method	NLL ↓	Brier ↓	ECE ↓	NLL ↓	Brier ↓	ECE ↓	NLL ↓	Brier ↓	ECE ↓	NLL ↓	Brier ↓	ECE ↓
LoRA (MAP)	1.76	0.52	1.68	1.75	0.52	1.48	5.16	0.97	0.82	5.16	0.97	1.60
LoRA + Temp	1.78	0.51	1.62	1.77	0.50	1.54	5.22	0.92	0.81	5.25	0.92	1.49
Dropout (S=4)	1.77	0.50	1.54	1.70	0.50	1.59	5.21	0.94	0.79	5.19	0.95	1.51
Bayesian-LoRA (S=1)	1.73	0.51	1.46	1.71	0.51	1.51	5.14	0.95	0.80	5.14	0.92	1.31
Bayesian-LoRA (S=2)	**1.70**	**0.48**	**1.36**	**1.66**	**0.48**	**1.41**	**5.13**	**0.91**	**0.79**	**5.12**	**0.90**	**1.26**

linear layers are replaced with our Bayesian implementation (details in §4). We evaluate on six commonsense reasoning benchmarks (dataset details in Appendix C). All comparison methods share the same data splits, compute budget, and decoding settings. We apply label smoothing (Szegedy et al., 2016) with smoothing factor $\epsilon = 0.1$ during training, which provides mild regularization and complements the Bayesian uncertainty modeling. Early stopping selects the checkpoint with the lowest validation NLL, following standard practice.[1] Each configuration is run with three

[1]NLL-based early stopping is applied uniformly to all methods; Bayesian-LoRA also achieves the highest accuracy on most

Table 3. Performance on the **MATH** dataset with large-scale models. We report Chain-of-Thought NLL (CoT-NLL), CoT Expected Calibration Error (CoT-ECE), and final answer accuracy. Training hyperparameters are in Table 16.

Model (Zero-shot)	Method	CoT-NLL ↓	CoT-ECE ↓	Answer Acc. ↑
Qwen2.5-14B-Instruct	Baseline FT	2.165	12.2	49.8
	Dropout (Gal & Ghahramani, 2016)	2.103	11.9	50.0
	Temp (Guo et al., 2017)	1.96	10.7	49.9
	LA (post-hoc) (Yang et al., 2024)	0.81	7.12	49.8
	BLoB (N=10) (Wang et al., 2024b)	1.21	8.41	47.2
	Bayesian-LoRA (N=4)	**0.513**	**5.81**	**51.1**
Qwen3-30B-A3B-Instruct-2507	Baseline FT	1.096	8.96	61.8
	Temp (Guo et al., 2017)	1.021	8.94	61.7
	LA (post-hoc) (Yang et al., 2024)	0.904	7.08	61.8
	BLoB (N=10) (Wang et al., 2024b)	0.993	7.15	60.4
	Bayesian-LoRA	**0.721**	**6.32**	**61.9**

random seeds; we report mean ± std for Accuracy (ACC ↑), Expected Calibration Error (ECE ↓; 15 bins), and Negative Log-Likelihood (NLL ↓).

5.1. In-Distribution Evaluation

Table 1 compares Bayesian-LoRA against eight baselines on six commonsense reasoning benchmarks. Bayesian-LoRA achieves the highest accuracy on five of six benchmarks, with improvements of up to $+2.9$ points over standard LoRA. It also obtains the best ECE on two benchmarks (WG-S with an 84% reduction over MAP, and WG-M) and the best NLL on four benchmarks. These improvements come from probabilistic training rather than post-hoc rescaling. Metric interpretations are in Appendix D.

Generative language modeling. Table 2 evaluates Bayesian-LoRA on generative language modeling (WikiText-2). Bayesian-LoRA (S=2) outperforms all baselines across NLL, Brier score, and ECE on both validation and test splits. The improvement is pronounced on the top-5% most uncertain tokens, where Bayesian-LoRA reduces test ECE from 1.60 (MAP) to 1.26. Unlike LA/LLLA, which require Hessian computation in generative settings, **Bayesian-LoRA** estimates uncertainty end-to-end during training.

5.2. Scaling to Larger Architectures

To demonstrate that Bayesian-LoRA's benefits extend beyond the 7B scale, we evaluate on mathematical reasoning (MATH) with **Qwen2.5-14B-Instruct** (a dense 14B model) and **Qwen3-30B-A3B-Instruct-2507** (a mixture-of-experts model with 30B total parameters but ≈ 3B active parameters per token).

benchmarks (Table 1).

Table 3 compares **Bayesian-LoRA** against Baseline FT, Dropout, Temperature Scaling, LA (post-hoc), and BLoB. On Qwen2.5-14B-Instruct, **Bayesian-LoRA** achieves CoT-NLL of 0.513 and CoT-ECE of 5.81, outperforming all baselines while improving accuracy to 51.1%. On Qwen3-30B-A3B, both NLL and ECE improve with no accuracy loss, confirming the $\mathcal{O}(rc)$ overhead remains negligible at scale.

5.3. Efficiency Analysis

Table 4 compares computational overhead, normalized to standard LoRA (MAP):

- **Training memory.** Deep Ensembles require $\sim 3\times$ peak memory to train multiple independent models, which becomes prohibitive for 30B+ LLMs. Bayesian-LoRA adds only $1.003\times$ peak memory, enabling uncertainty quantification under the strict memory budget of a single model.
- **Training time.** Bayesian-LoRA adds $1.229\times$ training time over MAP, compared to BBB ($4.19\times$), Dropout ($4\times$), and Deep Ensembles ($3\times$ for 3 members).
- **Inference.** Bayesian-LoRA supports two modes: (1) *deterministic mode* merges the posterior mean $W_{\text{merged}}=W_{\text{pre}}+T_r\mathbb{E}[U]T_c$ into the base weights with zero latency overhead, identical to standard LoRA; (2) *uncertainty mode* uses S samples when calibrated confidence estimates are required. With S=2, latency is $1.516\times$ MAP.
- **Parameters.** Only 0.42M additional parameters (4.9M vs. 4.48M), the smallest overhead among all Bayesian methods.

Table 4. Efficiency comparison normalized to standard LoRA (MAP) on WinoGrande-M. All baseline results are reproduced from open-source code.

Method	Trainable Parameters	Train time (×MAP)	Peak memory (×MAP)	Inference latency (×MAP)	Samples (Val)
MAP (LoRA) (Hu et al., 2022)	4.48M	1.00	1.00	1.00	1
Dropout (Gal & Ghahramani, 2016)	4.48M	≈ 4×	≈ 1×	≈ 4×	4
Ckpt Ens (3) (Huang et al., 2017)	1× MAP	≈ 1×	≈ 1×	≈ 3×	3
Deep Ens (3) (Lakshminarayanan et al., 2017)	3× MAP	≈ 3×	≈ 3×	≈ 3×	3
BBB (Blundell et al., 2015)	≈ 2× MAP	≈ 4.19×	≈ 1.05×	≈ 4.6×	4
BLoB (N=4) (Wang et al., 2024b)	≈1.5×MAP	≈ 1.11×	≈ 0.95×	≈ 6.3×	4
LLLA (post-hoc) (Yang et al., 2024)	4.48M + KFAC (≈0.36M)	≈ 1.052×	≈ 1.002×	≈ 1.9×	-
LA (post-hoc) (Yang et al., 2024)	4.48M + KFAC (≈4.98M)	≈ 1.117×	≈ 1.004×	≈ 4.36×	-
Bayesian-LoRA (End-to-End)	4.9M	≈ 1.229×	≈ 1.003×	≈ 1.516× ≈ 2.790×	2 4

Table 5. Ablation on flow depth L in the posterior transform T_ϕ (OBQA). Values are macro-averages (mean ± std over 3 seeds). Δ denotes the change from the L=1 baseline. Efficiency is relative to standard LoRA (MAP).

Flow depth L	**ACC** ↑		**ECE** ↓		**NLL** ↓		**Efficiency** (× MAP)	
	value	Δ vs. L=1	value	Δ vs. L=1	value	Δ vs. L=1	train time	peak mem.
0 (pure SGP)	79.0 ± 0.21	-2.6	5.8 ± 0.13	+0.1	0.58 ± 0.08	+0.01	1.19	1.002
1	81.6 ± 0.10	0.0	5.7 ± 0.20	0.0	0.57 ± 0.12	0.00	1.23	1.003
2	80.8 ± 0.14	-0.8	5.6 ± 0.09	-0.1	0.52 ± 0.06	-0.05	1.30	1.008
4	80.9 ± 0.08	-0.7	4.9 ± 0.03	-0.8	0.48 ± 0.13	-0.09	1.38	1.010

5.4. Ablation Study

Flow depth. Table 5 ablates the normalizing flow depth L on OBQA. Without any flow (L=0, pure SGP), accuracy drops by 2.6 points relative to L=1, confirming that the flow improves posterior expressiveness. A single flow layer (L=1) provides the best accuracy and a good accuracy–calibration trade-off. Deeper flows (L=2, 4) improve NLL and ECE at the cost of increased training time, but with diminishing gains. We adopt L=1 as the default. Figure 2 varies the inducing-point dimension r=c while keeping other hyperparameters fixed. The main experiments use r=c=9 to match the LoRA rank for a fair comparison. Increasing the inducing dimension improves calibration with diminishing returns beyond r=16; the parameter counts for each rank are compared in Appendix G, Table 16.

Empirical validation of MAP recovery (Corollary 4.1). Corollary 4.1 predicts that Bayesian-LoRA reduces to standard LoRA when the posterior collapses to a point mass and the conditional noise vanishes. We verify this empirically on OBQA by training Bayesian-LoRA with near-degenerate settings: λ_{init}=10^{-4}, $\lambda_{\max}$=10^{-4}, $\sigma_{U,\max}$=10^{-3}, and flow depth L=0 (identity transform). Table 7 compares this configuration against standard LoRA (MAP).

The degenerate configuration matches standard LoRA, confirming that LoRA lies on the boundary of the Bayesian-LoRA posterior family. The full model improves all metrics by leveraging calibrated uncertainty that the MAP limit discards.

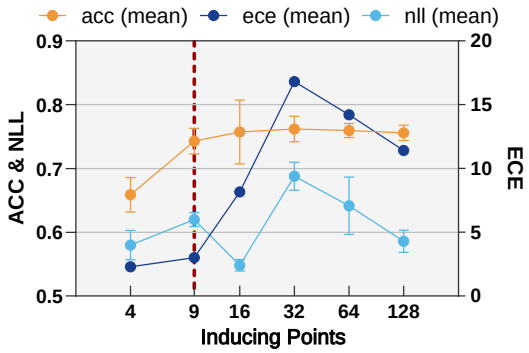

Figure 2. Ablation on inducing-point dimension $r = c$. Increasing the dimension improves calibration (lower ECE) with diminishing returns beyond r=16.

5.5. Out-of-Distribution Robustness

Table 6 evaluates robustness under distribution shift (small: ARC-C/E; large: CS, Eng, Law, Health from MMLU). Bayesian-LoRA achieves the best OOD accuracy on 5 of 6 shifted datasets. For ECE, LA (post-hoc) leads *under small shifts* (ID, ARC-C, ARC-E) via precise temperature tuning. *Under large shifts*, Bayesian-LoRA achieves the best ECE on 3 of 4 domains (CS, Law, Health); LA retains an advantage on Engineering. Post-hoc calibration learns a fixed rescaling that becomes stale under severe shift, while end-to-end training embeds uncertainty into the weights. Bayesian-LoRA achieves the best NLL on 6 of 7 sets.

Bayesian-LoRA and post-hoc methods are complementary:

Table 6. Robustness under distribution shift. We evaluate on the in-distribution OBQA test set and six out-of-distribution datasets spanning small shifts (ARC-C, ARC-E) and large shifts (CS, Eng, Law, Health from MMLU). Experimental settings follow Yang et al. (2024).

		ID	Smaller Distribution Shift		Larger Distribution Shift			
Metrics	Methods	OBQA	ARC-C	ARC-E	CS	Eng	Law	Health
ACC ↑	MAP (Hu et al., 2022)	78.7 ± 0.4	67.9 ± 1.4	77.7 ± 0.3	42.0 ± 3.2	41.2 ± 2.0	37.4 ± 0.4	48.3 ± 0.3
	Dropout (Gal & Ghahramani, 2016)	79.5 ± 0.2	67.7 ± 0.6	77.2 ± 0.6	41.9 ± 2.2	39.6 ± 1.7	37.9 ± 0.4	48.2 ± 0.9
	Ckpt Ens (Huang et al., 2017)	79.1 ± 0.2	67.9 ± 0.8	77.4 ± 0.8	41.1 ± 2.0	38.7 ± 1.2	37.7 ± 0.2	48.2 ± 0.6
	Temp (Guo et al., 2017)	77.7 ± 0.8	68.0 ± 0.2	76.7 ± 1.0	43.5 ± 0.9	$\mathbf{44.4 \pm 2.0}$	37.4 ± 0.1	47.7 ± 0.8
	BBB (Blundell et al., 2015)	77.0 ± 0.2	67.3 ± 1.2	75.8 ± 0.8	40.5 ± 0.3	36.7 ± 0.2	36.1 ± 0.2	47.6 ± 0.5
	BLoB(N=10) (Wang et al., 2024b)	81.5 ± 0.7	67.7 ± 1.1	76.3 ± 0.8	44.6 ± 0.4	42.3 ± 1.7	37.4 ± 1.2	47.3 ± 1.5
	LLLA (Yang et al., 2024)	78.7 ± 0.4	68.1 ± 0.0	78.1 ± 0.0	45.6 ± 0.0	38.9 ± 0.0	37.1 ± 0.0	48.5 ± 0.0
	LA (Yang et al., 2024)	78.9 ± 0.2	69.2 ± 0.0	78.5 ± 0.0	45.1 ± 0.0	39.1 ± 0.0	37.3 ± 0.0	49.1 ± 0.0
	Bayesian-LoRA (S=4) (End-to-End)	$\mathbf{81.6 \pm 0.1}$	$\mathbf{69.5 \pm 0.1}$	$\mathbf{78.9 \pm 0.2}$	$\mathbf{46.3 \pm 0.1}$	37.5 ± 0.2	$\mathbf{37.9 \pm 0.1}$	$\mathbf{50.6 \pm 0.1}$
ECE ↓	MAP (Hu et al., 2022)	16.1 ± 0.6	22.2 ± 1.2	15.8 ± 1.0	34.2 ± 3.1	38.4 ± 1.7	35.2 ± 0.7	34.2 ± 0.8
	Dropout (Gal & Ghahramani, 2016)	15.0 ± 0.4	21.4 ± 0.4	15.5 ± 1.0	33.7 ± 2.0	38.6 ± 2.9	34.2 ± 0.6	33.5 ± 0.2
	Ckpt Ens (Huang et al., 2017)	10.1 ± 0.3	17.7 ± 0.7	12.1 ± 0.6	29.1 ± 2.3	32.5 ± 1.8	32.1 ± 0.1	29.0 ± 0.3
	Temp (Guo et al., 2017)	7.20 ± 2.6	9.41 ± 3.5	6.33 ± 1.3	16.4 ± 5.5	16.1 ± 4.6	22.7 ± 5.8	17.4 ± 6.0
	BBB (Blundell et al., 2015)	11.3 ± 1.1	19.9 ± 0.7	13.4 ± 0.9	18.6 ± 1.6	22.4 ± 3.1	28.1 ± 2.5	27.4 ± 1.8
	BLoB (N=10) (Wang et al., 2024b)	3.77 ± 1.5	9.55 ± 0.4	5.48 ± 1.3	12.6 ± 1.7	22.3 ± 1.9	25.3 ± 2.1	16.4 ± 1.6
	LLLA (Yang et al., 2024)	15.8 ± 0.6	21.3 ± 0.0	14.8 ± 0.0	30.3 ± 0.0	39.7 ± 0.0	33.6 ± 0.0	33.5 ± 0.0
	LA (Yang et al., 2024)	$\mathbf{3.50 \pm 0.4}$	$\mathbf{5.50 \pm 0.0}$	$\mathbf{3.10 \pm 0.0}$	14.5 ± 0.0	$\mathbf{12.8 \pm 0.0}$	23.9 ± 0.0	17.6 ± 0.0
	Bayesian-LoRA (S = 4) (End-to-End)	5.70 ± 0.2	8.10 ± 0.1	5.21 ± 0.1	$\mathbf{11.1 \pm 0.0}$	20.4 ± 0.1	$\mathbf{16.5 \pm 0.0}$	$\mathbf{12.9 \pm 0.0}$
NLL ↓	MAP (Hu et al., 2022)	0.99 ± 0.05	1.30 ± 0.07	1.04 ± 0.10	1.90 ± 0.12	2.19 ± 0.15	2.12 ± 0.03	2.09 ± 0.08
	Dropout (Gal & Ghahramani, 2016)	0.95 ± 0.04	1.24 ± 0.06	1.01 ± 0.09	1.86 ± 0.10	2.14 ± 0.13	2.09 ± 0.02	2.05 ± 0.07
	Ckpt Ens (Huang et al., 2017)	0.68 ± 0.03	1.03 ± 0.03	0.80 ± 0.03	1.55 ± 0.04	1.72 ± 0.01	1.94 ± 0.01	1.74 ± 0.02
	Temp (Guo et al., 2017)	0.67 ± 0.02	0.90 ± 0.05	0.66 ± 0.01	1.31 ± 0.06	1.32 ± 0.07	1.65 ± 0.16	1.36 ± 0.10
	BBB (Blundell et al., 2015)	0.66 ± 0.05	1.06 ± 0.01	0.79 ± 0.02	1.49 ± 0.05	1.83 ± 0.08	1.65 ± 0.20	1.29 ± 0.12
	BLoB (N=10) (Wang et al., 2024b)	0.50 ± 0.01	0.83 ± 0.01	$\mathbf{0.60 \pm 0.01}$	1.38 ± 0.01	1.81 ± 0.10	2.01 ± 0.09	1.94 ± 0.08
	LLLA (Yang et al., 2024)	0.66 ± 0.02	0.88 ± 0.00	0.64 ± 0.00	1.28 ± 0.00	1.27 ± 0.00	1.49 ± 0.00	1.31 ± 0.00
	LA (Yang et al., 2024)	0.62 ± 0.01	0.85 ± 0.00	0.62 ± 0.00	1.26 ± 0.00	1.27 ± 0.00	1.68 ± 0.00	1.35 ± 0.00
	Bayesian-LoRA (S=4) (End-to-End)	$\mathbf{0.49 \pm 0.02}$	$\mathbf{0.82 \pm 0.04}$	0.80 ± 0.04	$\mathbf{1.22 \pm 0.04}$	$\mathbf{1.26 \pm 0.02}$	$\mathbf{1.42 \pm 0.04}$	$\mathbf{1.20 \pm 0.02}$

Table 7. MAP recovery validation on OBQA. Degenerate B-LoRA uses near-zero posterior variance and no flow, approximating the MAP limit of Corollary 4.1. Results are mean±std over 3 seeds.

Method	**ACC ↑**	**ECE ↓**	**NLL ↓**
LoRA (MAP)	77.70 ± 0.80	9.80 ± 1.00	0.71 ± 0.03
Degenerate B-LoRA	77.81 ± 0.71	9.77 ± 1.02	0.70 ± 0.02
Bayesian-LoRA (full)	81.60 ± 0.10	5.70 ± 0.20	0.49 ± 0.02

post-hoc excels when test data resembles the calibration set, while end-to-end Bayesian training is more robust under distributional mismatch. See Appendix A.7 for analysis and Appendix G for hyperparameter optimization.

6. Conclusion

We identified a structural isomorphism (shared bilinear functional form) between Kronecker-factored SGP posteriors and LoRA's factorization, with Bayesian-LoRA reducing to deterministic LoRA in the limit. Our method replaces LoRA's point estimate with a flow-augmented variational posterior for calibration-aware training. Across commonsense reasoning, language modeling, and math benchmarks at scales up to 14B (dense) and 30B (MoE), Bayesian-LoRA achieves competitive accuracy with improved NLL. ECE gains are largest under distribution shift; post-hoc methods remain effective for small shifts.

Limitations. Our per-layer inducing matrices are modeled independently; hierarchical priors could capture inter-layer correlations. The Bayesian optimization analysis (Table 14) shows accuracy-calibration trade-offs with objective-dependent optimal hyperparameters. Extensions to other modalities and instruction-tuning/RLHF remain future work, as do tighter theoretical bounds on the sparse inducing approximation.

Impact Statement

This work improves calibration and uncertainty quantification for fine-tuned LLMs. Well calibrated models are essential for safe deployment in high-stakes domains such as medical diagnosis and autonomous systems, helping users identify when model outputs may be unreliable. We do not foresee negative societal consequences unique to this work.

References

Blundell, C., Cornebise, J., Kavukcuoglu, K., and Wierstra, D. Weight uncertainty in neural network. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37, pp. 1613–1622. PMLR, 2015.

Burt, D. R., Rasmussen, C. E., and Van Der Wilk, M. Convergence of sparse variational inference in gaussian processes regression. *Journal of Machine Learning Research*, 21(131):1–63, 2020.

Chuang, Y.-N., Yu, L., Wang, G., Zhang, L., Liu, Z., Cai, X., Sui, Y., Braverman, V., and Hu, X. Confident or seek stronger: Exploring uncertainty-based on-device llm routing from benchmarking to generalization. *arXiv preprint arXiv:2502.04428*, 2025.

Daxberger, E., Kristiadi, A., Immer, A., Eschenhagen, R., Bauer, M., and Hennig, P. Laplace redux-effortless bayesian deep learning. In *Advances in neural information processing systems*, volume 34, pp. 20089–20103, 2021.

Desai, S. and Durrett, G. Calibration of pre-trained transformers. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 295–302, 2020.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. Qlora: Efficient finetuning of quantized llms. In *Advances in neural information processing systems*, volume 36, pp. 10088–10115, 2023.

Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., Hu, S., Chen, Y., Chan, C.-M., Chen, W., et al. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235, 2023.

Gal, Y. and Ghahramani, Z. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *Proceedings of The 33rd International Conference on Machine Learning*, volume 48, pp. 1050–1059. PMLR, 2016.

Graves, A. Practical variational inference for neural networks. In *Advances in neural information processing systems*, volume 24, pp. 2348–2356, 2011.

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70, pp. 1321–1330. PMLR, 2017.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021.

Houlsby, N., Giurgiu, A., Jastrzebski, S., Morrone, B., De Laroussilhe, Q., Gesmundo, A., Attariyan, M., and Gelly, S. Parameter-efficient transfer learning for nlp. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97, pp. 2790–2799. PMLR, 2019.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Hu, J., Xu, X., and Zou, Z. Lora-medsam: Efficient medical image segmentation. In *International Conference on Medical Imaging and Computer-Aided Diagnosis*, volume 1372, pp. 154–164. Springer, 2024.

Huang, G., Li, Y., Pleiss, G., Liu, Z., Hopcroft, J. E., and Weinberger, K. Q. Snapshot ensembles: Train 1, get m for free. In *International Conference on Learning Representations*, 2017.

Izmailov, P., Vikram, S., Hoffman, M. D., and Wilson, A. G. What are bayesian neural network posteriors really like? In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 4629–4640. PMLR, 2021.

Kim, Y., Jeong, H., Chen, S., Li, S. S., Lu, M., Alhamoud, K., Mun, J., Grau, C., Jung, M., Gameiro, R., et al. Medical hallucinations in foundation models and their impact on healthcare. *arXiv preprint arXiv:2503.05777*, 2025.

Kull, M., Perello Nieto, M., Kängsepp, M., Silva Filho, T., Song, H., and Flach, P. Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with dirichlet calibration. In *Advances in neural information processing systems*, volume 32, pp. 12316, 2019.

Kweon, W., Jang, S., Kang, S., and Yu, H. Uncertainty quantification and decomposition for llm-based recommendation. In *Proceedings of the ACM on Web Conference 2025*, pp. 4889–4901, 2025.

Lakshminarayanan, B., Pritzel, A., and Blundell, C. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in neural information processing systems*, volume 30, pp. 6402–6413, 2017.

Lester, B., Al-Rfou, R., and Constant, N. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2021.

Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 4582–4597, 2021.

Lin, M., Guan, S., Jing, W., Botterweck, G., and Patane, A. Stochastic weight sharing for bayesian neural networks. In *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, volume 258, pp. 4519–4527. PMLR, 2025a.

Lin, M., Patane, A., Jing, W., Guan, S., and Botterweck, G. Flow-induced diagonal gaussian processes. *arXiv preprint arXiv:2509.17153*, 2025b.

Lin, X., Wang, W., Li, Y., Yang, S., Feng, F., Wei, Y., and Chua, T.-S. Data-efficient fine-tuning for llm-based recommendation. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*, pp. 365–374, 2024.

Liu, H., Huang, H., Gu, X., Wang, H., and Wang, Y. On calibration of llm-based guard models for reliable content moderation. In *International Conference on Learning Representations*, 2024.

Liu, X., Chen, T., Da, L., Chen, C., Lin, Z., and Wei, H. Uncertainty quantification and confidence calibration in large language models: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 6107–6117, 2025.

Mai, Z., Chowdhury, A., Zhang, P., Tu, C.-H., Chen, H.-Y., Pahuja, V., Berger-Wolf, T., Gao, S., Stewart, C., Su, Y., et al. Fine-tuning is fine, if calibrated. In *Advances in neural information processing systems*, volume 37, pp. 136084–136119, 2024.

Malladi, S., Gao, T., Nichani, E., Damian, A., Lee, J. D., Chen, D., and Arora, S. Fine-tuning language models with just forward passes. In *Advances in neural information processing systems*, volume 36, pp. 53038–53075, 2023.

Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., and Paul, S. Peft: State-of-the-art parameter-efficient fine-tuning methods. https://github.com/huggingface/peft, 2022. Accessed: 2025-09-02.

Papamakarios, G., Pavlakou, T., and Murray, I. Masked autoregressive flow for density estimation. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Pfeiffer, J., Kamath, A., Rücklé, A., Cho, K., and Gurevych, I. Adapterfusion: Non-destructive task composition for transfer learning. In *Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pp. 487–503, 2021.

Quinonero-Candela, J. and Rasmussen, C. E. A unifying view of sparse approximate gaussian process regression. *Journal of machine learning research*, 6(Dec):1939–1959, 2005.

Ranković, B. and Schwaller, P. Gollum: Gaussian process optimized llms—reframing llm finetuning through bayesian optimization. In *ICLR 2025 Workshop on World Models: Understanding, Modelling and Scaling*, 2025.

Rezende, D. and Mohamed, S. Variational inference with normalizing flows. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 1530–1538. PMLR, 2015.

Ritter, H., Botev, A., and Barber, D. A scalable laplace approximation for neural networks. In *International Conference on Learning Representations*. International Conference on Representation Learning, 2018.

Ritter, H., Kukla, M., Zhang, C., and Li, Y. Sparse uncertainty representation in deep learning with inducing weights. *Advances in Neural Information Processing Systems*, 34:6515–6528, 2021.

Sankararaman, K. A., Wang, S., and Fang, H. Bayesformer: Transformer with uncertainty estimation. *arXiv preprint arXiv:2206.00826*, 2022.

Savage, T., Wang, J., Gallo, R., Boukil, A., Patel, V., Safavi-Naini, S. A. A., Soroush, A., and Chen, J. H. Large language model uncertainty proxies: discrimination and calibration for medical diagnosis and treatment. *Journal of the American Medical Informatics Association*, 32(1): 139–149, 2025.

Seeger, M. Gaussian processes for machine learning. *International journal of neural systems*, 14(02):69–106, 2004.

Shorinwa, O., Mei, Z., Lidard, J., Ren, A. Z., and Majumdar, A. A survey on uncertainty quantification of large language models: Taxonomy, open research challenges, and future directions. *ACM Computing Surveys*, 2025.

Snelson, E. and Ghahramani, Z. Sparse gaussian processes using pseudo-inputs. *Advances in neural information processing systems*, 18, 2005.

Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., and Wojna, Z. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2818–2826, 2016.

Titsias, M. Variational learning of inducing variables in sparse gaussian processes. In *Proceedings of the 12th International Conference on Artificial Intelligence and Statistics*, volume 5, pp. 567–574. PMLR, 2009.

Tu, J., Ji, W., Zhao, H., Zhang, C., Zimmermann, R., and Qian, H. Driveditfit: Fine-tuning diffusion transformers for autonomous driving data generation. *ACM Transactions on Multimedia Computing, Communications and Applications*, 21(3):1–29, 2025.

Wang, C. Calibration in deep learning: A survey of the state-of-the-art. *arXiv preprint arXiv:2308.01222*, 2023.

Wang, Y., Huang, Z., Liu, Q., Zheng, Y., Hong, J., Chen, J., Xiong, L., Gao, B., and Chen, H. Drive as veteran: Fine-tuning of an onboard large language model for highway autonomous driving. In *2024 IEEE Intelligent Vehicles Symposium (IV)*, pp. 502–508. IEEE, 2024a.

Wang, Y., Shi, H., Han, L., Metaxas, D., and Wang, H. Blob: Bayesian low-rank adaptation by backpropagation for large language models. In *Advances in neural information processing systems*, volume 37, pp. 67758–67794, 2024b.

Wu, S., Hadachi, A., Vivet, D., and Prabhakar, Y. This is the way: Sensors auto-calibration approach based on deep learning for self-driving cars. *IEEE Sensors Journal*, 21 (24):27779–27788, 2021.

Xu, R., Luo, F., Zhang, Z., Tan, C., Chang, B., Huang, S., and Huang, F. Raise a child in large language model: Towards effective and generalizable fine-tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 9514–9528, 2021.

Xue, B., Yu, J., Xu, J., Liu, S., Hu, S., Ye, Z., Geng, M., Liu, X., and Meng, H. Bayesian transformer language models for speech recognition. In *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 7378–7382. IEEE, 2021.

Yang, A. X., Robeyns, M., Wang, X., and Aitchison, L. Bayesian low-rank adaptation for large language models. In *International Conference on Learning Representations*, 2024.

Ye, F., Yang, M., Pang, J., Wang, L., Wong, D., Yilmaz, E., Shi, S., and Tu, Z. Benchmarking llms via uncertainty quantification. *Advances in Neural Information Processing Systems*, 37:15356–15385, 2024.

Yin, F., Ye, X., and Durrett, G. Lofit: Localized fine-tuning on llm representations. In *Advances in neural information processing systems*, volume 37, pp. 9474–9506, 2024.

Zhang, Q., Chen, M., Bukharin, A., He, P., Cheng, Y., Chen, W., and Zhao, T. Adaptive budget allocation for parameter-efficient fine-tuning. In *International Conference on Learning Representations*, 2023.

Zhou, N., Chen, J., and Huang, D. Dr-tune: Improving fine-tuning of pretrained visual models by distribution regularization with semantic calibration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1547–1556, 2023.

Zhou, Z., Sha, C., and Peng, X. On calibration of pre-trained code models. In *Proceedings of the IEEE/ACM 46th international conference on software engineering*, pp. 1–13, 2024.

A. Derivations for the Variational Sparse Inducing Weight Model

A.1. Model Specification and Joint Density

We define $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ as the layer weight matrix and $U \in \mathbb{R}^{r \times c}$ the low-dimensional inducing matrix with $r \ll d_{\text{out}}$ and $c \ll d_{\text{in}}$. We place a Gaussian (equivalently, matrix-normal) prior on U:

$$p(U) \;=\; \mathcal{N}\big(\mathrm{vec}(U) \mid \mathbf{0},\, K_U\big),\; K_U \;=\; K_c \otimes K_r \quad (16)$$

which is equivalent to $U \sim \mathcal{MN}(\mathbf{0}, K_r, K_c)$. Given U, the conditional distribution of W is Gaussian

$$p(W \mid U) \;=\; \mathcal{N}\big(W \mid M_W(U),\, \lambda^2 \Sigma_W\big) \quad (17)$$

where $\lambda > 0$ is a scale parameter and $M_W(U)$ is linear in U with

$$K_r \;=\; Z_r Z_r^\top + D_r^2, \qquad K_c \;=\; Z_c Z_c^\top + D_c^2 \quad (18)$$

The likelihood factorizes over data $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N$ as $p(\mathcal{D} \mid W) = \prod_{n=1}^N p(y_n \mid f(x_n; W))$. The joint density is:

$$p(U, W, \mathcal{D}) \;=\; p(U)\, p(W \mid U)\, p(\mathcal{D} \mid W) \quad (19)$$

A.2. Variational Family and Factorization

We posit a Gaussian variational posterior on U,

$$q(U) \;=\; \mathcal{N}\big(\mathrm{vec}(U) \mid \mathbf{m},\, \mathbf{S}\big) \quad (20)$$

and keep the model conditional $p(W \mid U)$ inside the variational family:

$$q(U, W) \;=\; q(U)\, p(W \mid U) \quad (21)$$

Marginalizing U gives the variational distribution over W:

$$q(W) \;=\; \int p(W \mid U)\, q(U)\, \mathrm{d}U \quad (22)$$

A.3. ELBO Derivation

Starting from $\log p(\mathcal{D}) = \log \int p(U, W, \mathcal{D})\, \mathrm{d}U\, \mathrm{d}W$ and inserting $q(U, W)$ (Ritter et al., 2021):

$$\log p(\mathcal{D}) = \log \int q(U, W)\, \frac{p(U)\, p(W \mid U)\, p(\mathcal{D} \mid W)}{q(U)\, p(W \mid U)}\, \mathrm{d}U\, \mathrm{d}W \quad (23)$$

Jensen's inequality yields the ELBO

$$\mathcal{L} = \mathbb{E}_{q(U)p(W|U)}\Big[\log p(\mathcal{D} \mid W)\Big] - \mathbb{E}_{q(U)}\Big[\log \tfrac{q(U)}{p(U)}\Big] \quad (24)$$

i.e.

$$\mathcal{L} \;=\; \mathbb{E}_{q(W)}\Big[\log p(\mathcal{D} \mid W)\Big] - \mathrm{KL}\big(q(U) \,\|\, p(U)\big) \quad (25)$$

Equivalently, by adding and subtracting $\mathbb{E}_{q(U)}[\log p(W \mid U)]$,

$$\begin{aligned}\mathcal{L} \;=\; & \mathbb{E}_{q(W)}\Big[\log p(\mathcal{D} \mid W)\Big] - \mathrm{KL}\big(q(U) \,\|\, p(U)\big) \\ & - \mathbb{E}_{q(U)}\Big[\mathrm{KL}\big(q(W \mid U) \,\|\, p(W \mid U)\big)\Big]\end{aligned} \quad (26)$$

which matches the presentation in the main text.

A.4. Closed-Form Terms

KL between Gaussians for $q(U)$ and $p(U)$. Define $d_U = rc$ and denote $\mathbf{m} \in \mathbb{R}^{d_U}$, $\mathbf{S} \in \mathbb{R}^{d_U \times d_U}$, and $K_U = K_c \otimes K_r$. Then

$$\begin{aligned}\mathrm{KL}\big(q(U) \,\|\, p(U)\big) = \frac{1}{2}\Big(\mathrm{tr}(K_U^{-1}\mathbf{S}) + \\ \mathbf{m}^\top K_U^{-1}\mathbf{m} - d_U + \log \tfrac{|K_U|}{|\mathbf{S}|}\Big)\end{aligned} \quad (27)$$

Using $\log |\mathbf{K}_c \otimes \mathbf{K}_r| = c \log |\mathbf{K}_r| + r \log |\mathbf{K}_c|$ simplifies evaluation. In the whitened case with $p(U) = \mathcal{N}(\mathbf{0}, \mathbf{I}_{d_U})$,

$$\begin{aligned}\mathrm{KL}\big(q(U) \,\|\, p(U)\big) = \\ \frac{1}{2}\Big(\mathrm{tr}(\mathbf{S}) + \mathbf{m}^\top \mathbf{m} - d_U - \log |\mathbf{S}|\Big)\end{aligned} \quad (28)$$

Conditional KL for $q(W \mid U)$ vs $p(W \mid U)$. Both share the same mean $M_W(U)$. The variational conditional has covariance $\Sigma_q = \lambda^2 \Sigma_W$ (from Eq. (17)), and the prior conditional has covariance $\Sigma_p = \Sigma_W$. Set $d_W = d_{\text{out}} d_{\text{in}}$. Then

$$\begin{aligned}&\mathrm{KL}\big(q(W \mid U) \,\|\, p(W \mid U)\big) \\ &= \frac{1}{2}\Big(\mathrm{tr}(\Sigma_p^{-1}\Sigma_q) - d_W + \log \tfrac{|\Sigma_p|}{|\Sigma_q|}\Big) \\ &= \frac{1}{2}\Big(\lambda^2 d_W - d_W - 2 d_W \log \lambda\Big) \\ &= \frac{d_W}{2}\big(\lambda^2 - 1 - 2\log \lambda\big)\end{aligned} \quad (29)$$

This is independent of U and thus the expectation $\mathbb{E}_{q(U)}[\cdot]$ in (26) is trivial.

A.5. Form of $q(W)$: Mean and Covariance

Vectorizing W and using $\mathrm{vec}(T_r U T_c) = (T_c^\top \otimes T_r)\, \mathrm{vec}(U)$, one obtains

$$\begin{aligned}\mathbb{E}_q[\,\mathrm{vec}(W)\,] \;&=\; (T_c^\top \otimes T_r)\, \mathbf{m}, \\ \mathbb{E}_q[\,W\,] \;&=\; T_r\, M\, T_c\end{aligned} \quad (30)$$

where M is the matricized version of $\mathbf{m}$. Moreover, since $\Sigma_q(W \mid U) = \lambda^2 \Sigma_W$ is independent of U,

$$\mathrm{Cov}_q[\,\mathrm{vec}(W)\,] = \lambda^2 \Sigma_W \;+\; (T_c^\top \otimes T_r)\, \mathbf{S}\, (T_c \otimes T_r^\top) \quad (31)$$

Algorithm 1 Bayesian-LoRA (replace A, B): One Training Step

Input: Batch $\mathcal{B}$, target layers $\mathcal{L}$, rank r, scale α, MC samples S, noise scale λ, flow T_ϕ, base distribution q_0, per-layer covariance factors $(K_r^{A/B}, K_c^{A/B})$ and projectors $(T_r^{A/B}, T_c^{A/B})$
Precompute: For each layer $\ell \in \mathcal{L}$, compute:
  $T_r^A = (Z_r^A)^\top (K_r^A)^{-1}, \quad T_c^A = (K_c^A)^{-1} Z_c^A$ ▷ Projection operators for A, Eq. (7)
  $T_r^B = (Z_r^B)^\top (K_r^B)^{-1}, \quad T_c^B = (K_c^B)^{-1} Z_c^B$ ▷ Projection operators for B, Eq. (7)
  $\Sigma_{A,\ell}^{1/2}, \Sigma_{B,\ell}^{1/2}$ ▷ Covariance factors, Eq. (5)
for $\ell \in \mathcal{L}$ **do**
  Sample inducing variables: $U_0^{(1)}, \dots, U_0^{(S)} \sim q_0(U_0)$ ▷ Base Gaussian, Eq. (13)
  Apply flow transform: $U^{(s)} \leftarrow T_\phi(U_0^{(s)})$ for $s = 1, \dots, S$ ▷ Normalizing flow, Eq. (13)
  for $s = 1$ to S **do**
    Compute conditional means:
      $\bar{A}_\ell^{(s)} = T_{r,\ell}^A \, U^{(s)} \, T_{c,\ell}^A$ ▷ Mean of A given U, Eq. (8)
      $\bar{B}_\ell^{(s)} = T_{r,\ell}^B \, U^{(s)} \, T_{c,\ell}^B$ ▷ Mean of B given U, Eq. (8)
    Sample Gaussian noise: $\varepsilon_A^{(s)}, \varepsilon_B^{(s)} \sim \mathcal{N}(0, I)$
    Add conditional variance:
      $A_\ell^{(s)} = \bar{A}_\ell^{(s)} + \lambda \, \Sigma_{A,\ell}^{1/2} \varepsilon_A^{(s)}$ ▷ Sample from $p(W_A \mid U)$, Eq. (5)
      $B_\ell^{(s)} = \bar{B}_\ell^{(s)} + \lambda \, \Sigma_{B,\ell}^{1/2} \varepsilon_B^{(s)}$ ▷ Sample from $p(W_B \mid U)$, Eq. (5)
    Form LoRA update: $\Delta W_\ell^{(s)} = \frac{\alpha}{r} B_\ell^{(s)} A_\ell^{(s)}$ ▷ Low-rank update, Eq. (1)
    Effective weight: $W_{\text{eff},\ell}^{(s)} = W_{\text{pre},\ell} + \Delta W_\ell^{(s)}$ ▷ Eq. (2)
  end for
end for
Compute ELBO:
  Likelihood: $\mathcal{L}_{\text{data}} = \frac{1}{S} \sum_{s=1}^{S} \log p(\mathcal{B} \mid \{W_{\text{eff},\ell}^{(s)}\}_{\ell \in \mathcal{L}})$ ▷ Expected log-likelihood, Eq. (15)
  KL (inducing): $\mathrm{KL}_U = \mathrm{KL}(q_\phi(U) \,\|\, p(U))$ ▷ KL in U-space, Eqs. (14)–(15)
  KL (conditional): $\mathrm{KL}_W = \frac{D}{2}(\lambda^2 - 1 - 2\log\lambda)$ where $D = \sum_{\ell \in \mathcal{L}} d_{\text{out},\ell} \times d_{\text{in},\ell}$ ▷ Closed-form, Eq. (29)
  $\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\text{data}} - \mathrm{KL}_U - \mathrm{KL}_W$ ▷ Eq. (15)
Output: $\mathcal{L}_{\text{ELBO}}$ for gradient-based optimization

Thus $q(W)$ is Gaussian with the above mean and covariance whenever $q(U)$ is Gaussian and $M_W(U)$ is linear in U.

Finally,

$$\log p(\mathcal{D}) \;=\; \mathcal{L} \;+\; \mathrm{KL}\big(q(U) \,\|\, p(U \mid \mathcal{D})\big), \tag{32}$$

So maximizing $\mathcal{L}$ minimizes the divergence from the variational posterior to the exact posterior over U.

Proposition A.1 (Details of U-space-independent KL)**.** *Let $U \in \mathbb{R}^{d_U}$ denote the inducing variables (with $d_U = rc$) with a Gaussian prior $p(U) = \mathcal{N}(\mu_p, \Sigma_p)$ and variational posterior $q_\psi(U) = \mathcal{N}(\mu_q, \Sigma_q)$. Let $T_\phi : \mathbb{R}^{d_U} \to \mathbb{R}^{d_U}$ be an invertible C^1 map (the adapter flow) with Jacobian $J_{T_\phi}(U)$, and define the LoRA parameters as the deterministic pushforward $\Delta W = T_\phi(U)$. For data $\mathcal{D}$ with likelihood $p(\mathcal{D} \mid \Delta W)$, the ELBO in U-space*

$$\begin{aligned}\mathcal{L}(\phi, \psi) = \mathbb{E}_{q_\psi(U)}\big[\log p(\mathcal{D} \mid T_\phi(U))\big] \\ - \mathrm{KL}\big(q_\psi(U) \,\|\, p(U)\big)\end{aligned} \tag{33}$$

is equivalent to the ELBO in ΔW-space,

$$\begin{aligned}\mathcal{L}(\phi, \psi) = \mathbb{E}_{q_\phi(\Delta W)}\big[\log p(\mathcal{D} \mid \Delta W)\big] \\ - \mathrm{KL}\big(q_\phi(\Delta W) \,\|\, p_\phi(\Delta W)\big)\end{aligned} \tag{34}$$

where $q_\phi(\Delta W) := T_{\phi\#} q_\psi$ and $p_\phi(\Delta W) := T_{\phi\#} p$ are pushforwards under T_ϕ. Moreover, the KL term is invariant under T_ϕ:

$$\mathrm{KL}\big(q_\phi(\Delta W) \,\|\, p_\phi(\Delta W)\big) = \mathrm{KL}\big(q_\psi(U) \,\|\, p(U)\big) \tag{35}$$

In particular, when q_ψ and p are Gaussian, the KL admits a closed form:

$$\begin{aligned}\mathrm{KL}\big(\mathcal{N}(\mu_q, \Sigma_q) \,\|\, \mathcal{N}(\mu_p, \Sigma_p)\big) = \frac{1}{2}\Bigg(\mathrm{tr}(\Sigma_p^{-1}\Sigma_q) \\ + (\mu_p - \mu_q)^\top \Sigma_p^{-1} (\mu_p - \mu_q) \\ - d_U + \log \frac{\det \Sigma_p}{\det \Sigma_q}\Bigg)\end{aligned} \tag{36}$$

Proof. Since $\Delta W = T_\phi(U)$ is a deterministic, invertible change of variables, the data term satisfies $\mathbb{E}_{q_\psi(U)}[\log p(\mathcal{D} \mid$

$T_\phi(U))] = \mathbb{E}_{q_\phi(\Delta W)}[\log p(\mathcal{D} \mid \Delta W)]$. For the KL term, write the pushforward densities via change of variables: $q_\phi(\Delta W) = q_\psi(U)\,|\det J_{T_\phi}(U)|^{-1}$ and $p_\phi(\Delta W) = p(U)\,|\det J_{T_\phi}(U)|^{-1}$ with $\Delta W = T_\phi(U)$. Then

$$\begin{aligned}\mathrm{KL}(q_\phi \| p_\phi) &= \int q_\phi(\Delta W) \log \frac{q_\phi(\Delta W)}{p_\phi(\Delta W)}\, \mathrm{d}\Delta W \\ &= \int q_\psi(U) \log \frac{q_\psi(U)}{p(U)}\, \mathrm{d}U = \mathrm{KL}(q_\psi \| p)\end{aligned} \tag{37}$$

where the Jacobian determinants cancel exactly. Combining the two parts establishes the equivalent ELBO forms (33)–(34) and the invariance (35); the KL does not depend on ΔW nor on T_ϕ. When q_ψ and p are Gaussian, (36) follows from the standard closed-form KL between multivariate Gaussians. □

A.6. Interpretation of ELBO Terms

The ELBO in Eq. (15) consists of three terms with the following interpretations:

1. **Expected log-likelihood.** The first term $\mathbb{E}_{U_0 \sim q_0,\, \epsilon}[\log p(\mathcal{D} \mid W)]$ represents the expected data fit. This is approximated by Monte Carlo sampling: we draw S inducing samples from q_0, project each through the conditional mean and add scaled Gaussian noise (controlled by λ) to obtain weight realizations, then average the log-likelihoods over these samples.

2. **KL over inducing variables.** The second term

$$\begin{aligned}\mathbb{E}_{U_0 \sim q_0}\Big[\log q_0(U_0) - \log\big|\det J_{T_\phi}(U_0)\big| \\ - \log p\big(T_\phi(U_0)\big)\Big]\end{aligned} \tag{38}$$

   computes $\mathrm{KL}(q_\phi(U) \,\|\, p(U))$ via the flow density (using the change-of-variables formula). By Proposition 3.1, this KL is invariant under the transformation T_ϕ, meaning it equals the KL in ΔW-space. Therefore, we may optimize in U-space while inducing a consistent posterior over weights.

3. **Conditional KL (closed-form).** The third term $\frac{D}{2}(\lambda^2 - 1 - 2\log\lambda)$ represents $\mathrm{KL}(q(W \mid U) \,\|\, p(W \mid U))$. Since $q(W \mid U)$ and $p(W \mid U)$ share the same conditional mean and differ only in their scale parameters ($\lambda\sigma$ vs. σ), the conditional KL reduces to this closed form, which is *independent of* U and requires no sampling or Hessian computation. The derivation is provided in Appendix A.5.

A.7. Analysis of Out-of-Distribution Robustness

Table 6 evaluates robustness under distribution shift, covering small shifts (ARC-C, ARC-E) and large shifts (CS, Eng, Law, Health from MMLU). We highlight three key findings:

Accuracy under shift. Bayesian-LoRA achieves the best OOD accuracy on 5 of 6 shifted datasets (ARC-C, ARC-E, CS, Law, Health) and the best ID accuracy (OBQA). The largest OOD accuracy gain is $+2.3$ points on Health over the next-best method (LA). This suggests that the end-to-end Bayesian training acts as a form of distributional regularization that prevents overfitting to the in-distribution data.

Calibration under shift. While LA achieves the best ECE under small shifts (ID and ARC-C/E), Bayesian-LoRA achieves the best ECE on three of four large-shift domains: CS (11.1), Law (16.5), and Health (12.9). However, LA retains an advantage on Engineering (12.8 vs. 20.4), indicating that the relative benefit of end-to-end Bayesian training varies across OOD domains. This is a critical distinction: post-hoc methods calibrate well on data similar to the training set but degrade under severe distribution shift, whereas end-to-end Bayesian training produces more robust uncertainty estimates.

Likelihood under shift. Bayesian-LoRA achieves the best NLL on 6 of 7 evaluation sets (all except ARC-E OOD). The NLL improvements under large shifts are substantial: e.g., 1.20 vs. 1.29 (BBB) on Health, 1.42 vs. 1.49 (LLLA) on Law. Combined with the efficiency results (Table 4), Bayesian-LoRA achieves OOD performance comparable to or better than computationally heavy ensembles at a fraction of the cost.

B. Effect of Monte Carlo Samples on Accuracy and Uncertainty

In this section, we analyze how the number of Monte Carlo samples S affects predictive accuracy and uncertainty for the in-distribution (ID) ARC dataset and the out-of-distribution (OOD) OBQA dataset at a fixed checkpoint. For each S, we report accuracy (ACC), negative log-likelihood (NLL), expected calibration error (ECE; 15 bins), and the average per-batch inference time. Figure 3 shows the relative changes in ACC, NLL, ECE and inference time with respect to $S = 1$ for both ARC (ID) and OBQA (OOD). As S increases, the inference time grows almost linearly and reaches close to an order-of-magnitude increase between $S = 1$ and $S = 10$. On the ARC (ID) data, ACC, NLL and ECE vary only within a very narrow range once $S \geq 2$, and mostly fluctuate rather than improving systematically. On the OBQA (OOD) data, larger S yields slightly lower NLL/ECE and marginally more stable accuracy, consistent with reduced Monte Carlo noise in the predictive distribution. However, the gains beyond $S = 2$–4 remain small compared to the additional latency, which supports our practical recommendation of using $S = 2$–4 as a good trade-off between uncertainty quality and computational cost.

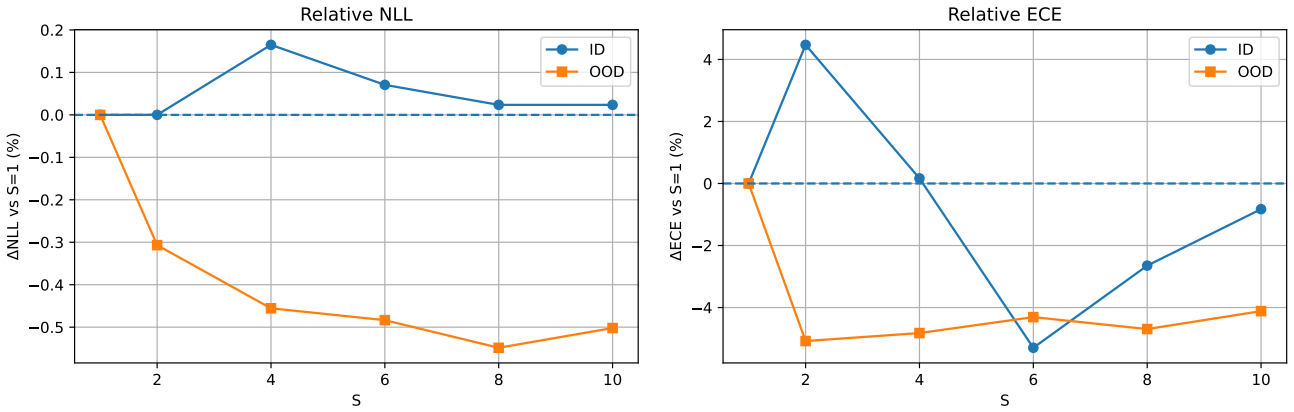

Figure 3. Relative changes in ACC, NLL, ECE and inference time with respect to $S = 1$ for the ID ARC dataset and the OOD OBQA dataset at a fixed checkpoint.

Table 8. Effect of the number of Monte Carlo samples S on OOD performance on OBQA at a fixed checkpoint. Time denotes average per-batch inference time in seconds.

S	NLL ↓	ACC ↑	ECE ↓	Time (s) ↓
1	1.0756	0.6333	0.1555	2.3997
2	1.0723	0.6354	0.1476	4.7858
3	1.0716	0.6333	0.1516	7.1729
4	1.0707	0.6354	0.1480	9.5596
5	1.0708	0.6313	0.1521	11.9460
6	1.0704	0.6354	0.1488	14.3300
7	1.0699	0.6333	0.1503	16.7140
8	1.0697	0.6354	0.1482	19.0961
9	1.0697	0.6354	0.1509	21.4805
10	1.0702	0.6354	0.1491	23.8661

Table 9. Effect of the number of Monte Carlo samples S on ID performance on ARC at the same checkpoint. Time denotes average per-batch inference time in seconds.

S	NLL ↓	ACC ↑	ECE ↓	Time (s) ↓
1	0.4245	0.8697	0.0604	0.7610
2	0.4245	0.8662	0.0631	1.5253
3	0.4253	0.8662	0.0644	2.2888
4	0.4252	0.8662	0.0605	3.0507
5	0.4252	0.8680	0.0584	3.8133
6	0.4248	0.8680	0.0572	4.5761
7	0.4246	0.8662	0.0606	5.3384
8	0.4246	0.8680	0.0588	6.1010
9	0.4249	0.8680	0.0572	6.8632
10	0.4246	0.8680	0.0599	7.6256

C. Datasets and Preprocessing

This appendix details the six benchmarks used in our evaluation, together with the scoring rules, prompting templates, and implementation choices shared across datasets.

C.1. Task Overview

All tasks are cast as *closed-set prediction* to avoid generation artifacts. For multiple-choice datasets (ARC-C/E, OBQA) and cloze-style coreference (WinoGrande S/M), we compute option probabilities without free decoding; for BoolQ, we map to a binary verbalizer. Table 10 summarizes formats.

Common evaluation protocol. Unless otherwise noted: (i) we use the official training/validation/test splits; (ii) Accuracy (ACC ↑), Expected Calibration Error (ECE ↓, 15 bins on $[0, 1]$ using the model's predicted probability for the chosen label), and Negative Log-Likelihood (NLL ↓) are reported; (iii) probabilities are computed from normalized option log-likelihoods as detailed below; (iv) casing and punctuation are preserved.

Tokenizer and limits. We use the backbone's SentencePiece tokenizer. Inputs are truncated to a maximum of $L_{\max}$ tokens (default 1024) by trimming long contexts first while keeping all answer options intact. Padding is on the right; the BOS/EOS usage follows the backbone defaults.

Scoring rule for multiple-choice/cloze. Given a prefix prompt x and an option string y_j tokenized as $(y_{j,1}, \ldots, y_{j,T_j})$, we score

$$s_j = \frac{1}{T_j} \sum_{t=1}^{T_j} \log p_\theta(y_{j,t} \mid x, y_{j,<t})$$

i.e., *length-normalized* log-likelihood to mitigate option-length bias. Predicted label $\hat{y} = \arg\max_j s_j$; option probabilities are $\pi_j \propto \exp(s_j)$ and are used for ECE/NLL. NLL is $-\log \pi_{y^\star}$ for gold label $y^\star$. In case of ties, we choose the option with the larger unnormalized (sum) log-likelihood, then lexicographically.

Table 10. Task formats and primary metrics.

Dataset	Task type	Candidates	Primary metric(s)
WinoGrande-S/M (WG-S/M)	Cloze coreference (2-way)	2	ACC, ECE, NLL
ARC-Challenge (ARC-C)	Multiple-choice science QA	3–5 (mostly 4)	ACC, ECE, NLL
ARC-Easy (ARC-E)	Multiple-choice science QA	3–5 (mostly 4)	ACC, ECE, NLL
OpenBookQA (OBQA)	Multiple-choice science QA	4	ACC, ECE, NLL
BoolQ	Yes/No reading comprehension	2	ACC, ECE, NLL

C.2. Prompting Templates

To minimize prompt sensitivity, we use deterministic templates without instructions or chain-of-thought. Placeholders are written as `<...>`.

WinoGrande (S/M). Each instance contains a sentence with a blank and two candidate fillers.

```
Sentence: <sentence>
Option A: <sentence with option1>
Option B: <sentence with option2>
```

We compare the completion likelihoods as in the scoring rule.

ARC-C / ARC-E.

```
Question: <question>
Options:
A. <choice A>
B. <choice B>
C. <choice C>
D. <choice D> [E. <choice E> if
present]
Answer:
```

Each option is scored by appending its text after `Answer:`.

OpenBookQA (OBQA). Same as ARC; four options (A–D). We omit the provided "facts" in the main results for parity.

BoolQ. Binary QA with verbalizers `Yes`/`No`.

```
Passage: <passage>
Question: <question>
Answer:
```

We score `Yes` and `No` as the two candidates.

C.3. Preprocessing and Normalization

- **Unicode/whitespace.** Normalize Unicode quotes/dashes; strip leading/trailing whitespace; collapse repeated spaces inside fields without altering semantics.
- **Deduplication.** Remove exact duplicate training examples (rare in these corpora); evaluation splits remain untouched.
- **Invalid items.** For ARC items with empty or malformed options, we drop the instance *from training only* and keep evaluation intact; such cases are logged ($< 0.1\%$ in our runs).
- **Context truncation.** When sequences exceed $L_{\max}$, we retain the full question and all options and truncate supporting passages from the left (BoolQ) or auxiliary fields (if used), preserving the end of the passage, which often contains the answer evidence.
- **Label integrity.** All label letters (A/B/...) are taken from the official annotations; we do not remap or reorder options.

C.4. Dataset-specific Notes

WinoGrande S/M. We use the official S and M partitions. Following common practice, we evaluate on the validation set for early stopping and report test numbers using the official held-out split when available. No external coreference resources are used.

ARC-Challenge / ARC-Easy. We use the AI2 ARC v1 format. Some questions have three or five options; our scoring rule handles variable cardinality. We do not incorporate retrieval or external knowledge in the main comparison.

OpenBookQA. We use the main OBQA v1.1 multiple-choice set. The "open book" facts are omitted in the main results for parity; adding them can increase accuracy, but does not change the calibration trends observed.

BoolQ. We keep case and punctuation in passages. Extremely long passages are truncated from the left to respect $L_{\max}$ while keeping the full question. Verbalizers are fixed as `Yes`/`No`; alternative synonyms (e.g., `True`/`False`) yield similar results.

D. Metrics for Uncertainty Quantification

NLL and ECE are two common metrics for quantifying model uncertainty. We note that ECE with a fixed number of bins provides a coarse summary of calibration. We use 15-bin ECE throughout for consistency with prior work (Yang et al., 2024; Guo et al., 2017).

Expected Calibration Error (ECE). We compute ECE over $B = 15$ equal-width bins on $[0, 1]$ for the predicted confidences c_i. Let $I_b = ((b-1)/B,\ b/B]$ and $B_b = \{\, i : c_i \in I_b \,\}$. Then:

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{|B_b|}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right| \tag{39}$$

$$\mathrm{acc}(B_b) = \frac{1}{|B_b|} \sum_{i \in B_b} \mathbf{1}\{\hat{y}_i = y_i\} \tag{40}$$

$$\mathrm{conf}(B_b) = \frac{1}{|B_b|} \sum_{i \in B_b} c_i,\ c_i = \max_k p_\theta(y{=}k \mid x_i) \tag{41}$$

Negative Log-Likelihood (NLL). For instance i with gold label y_i and predicted distribution $\hat{y}_i$,

$$\mathrm{NLL} = -\frac{1}{N} \sum_i \log P(\hat{y}_i = y_i) \tag{42}$$

For datasets with variable option counts, $\hat{y}_i$ is always the softmax of length-normalized scores across the *present* options.

E. Hyperparameters

Hyperparameters are crucial for reproducibility; therefore, we list all hyperparameters used in **Bayesian-LoRA**.

Table 11 lists all hyperparameters. We have also provided additional adjustable hyperparameters in the configuration to offer flexibility for reproducibility.

Also, the core settings of the Sparse Gaussian Process are listed in Table 12. Inducing rows and columns are set to 9, corresponding to the same parameter level as LoRA. The Q and K matrices of attention and the output linear layers are replaced by the Bayesian framework (SGP).

F. Out-of-Distribution Dataset

Following (Yang et al., 2024), we used the Computer Science (CS), Engineering (Eng), Law, and Health subsets of the MMLU dataset as out-of-distribution data to evaluate the robustness of **Bayesian-LoRA**. Table 13 summarizes the MMLU subjects used for OOD evaluation. Following the HuggingFace description of the MMLU dataset (Hendrycks et al., 2021), we split college computer science, computer security, high school computer science, and machine learning into the CS category, electrical engineering into the Eng category, international law, jurisprudence, and professional law into the Law category, and anatomy, clinical knowledge, college medicine, human aging, nutrition, professional medicine, and virology into the Health category.

G. Constrained Bayesian Optimization

In this section, we demonstrate the corresponding results for all datasets. Figure 4 presents the relationship of Negative Log-Likelihood and Expected Calibration Error with optima choice explicitly marked.

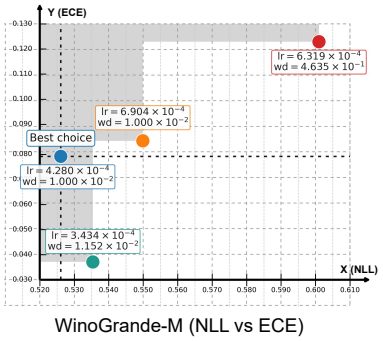

Figure 4. Pareto analysis on the WinoGrande-M dataset (NLL vs ECE). Each point denotes a hyperparameter pair $(\mathrm{lr}, \mathrm{wd})$. Gray regions show dominated solutions, and the cross marks the "Best choice" near the Pareto front.

We additionally present the ARC-Easy Pareto charts in Figure 5, using the same visualization scheme as WinoGrande-M (left: ACC vs. NLL; right: ACC vs. ECE).

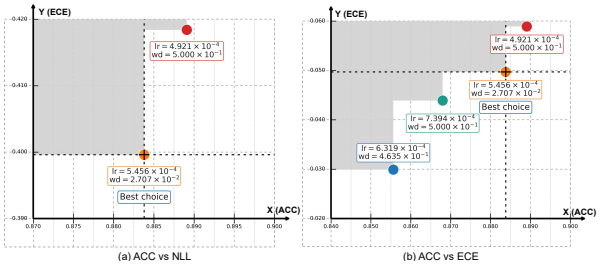

Figure 5. Pareto analysis on the ARC-Easy dataset (ACC vs NLL and ACC vs ECE). Each point denotes a hyperparameter pair $(\mathrm{lr}, \mathrm{wd})$. Gray regions show dominated solutions, and the cross marks the "Best choice" near the Pareto front.

Table 15 lists non-dominated candidates (w.r.t. ACC↑, NLL↓, ECE↓) returned by constrained BO for all datasets.

We select the final operating point by proximity to the empirical Pareto front (ties broken by lower NLL).

We tune learning rate η and weight decay λ_{wd} on a bounded

Table 11. Hyperparameters used in our experiments.

Hyper-parameter	**Value**
Optimizer	AdamW
Learning rate	5×10^{-4}
Betas	$(0.9,\ 0.999)$
Epsilon (ϵ)	1×10^{-5}
Weight decay	0.1
Scheduler	MultiStepLR (milestones = [4, 6], $\gamma = 0.1$)
Epochs	10
Batch size (train)	16 (WG-S/M, BoolQ), 8 (ARC-C/E, OBQA)
Batch size (eval)	32 (all datasets)
MC samples (S)	2 (default, can be set in cfg.eval)
KL scaling	$0.2/\text{steps\_per\_epoch}$
Evaluation frequency	Every 2 epochs

Table 12. Inducing hyper-parameters.

Hyper-parameter	**Value**
inducing_rows	9
inducing_cols	9
whitened_u	True
q_inducing	diagonal
learn_lambda	True
init_lambda	0.001
max_lambda	0.03
max_sd_u	0.1
cache_cholesky	True
prior_sd	0.1
sqrt_width_scaling	True
key_layers	{q_proj, k_proj, lm_head}

domain $\mathcal{X} \subset \mathbb{R}^2$:

$$\min_{\mathbf{x}\in\mathcal{X}} \mathbf{f}(\mathbf{x}) = \big(f_1(\mathbf{x}), f_2(\mathbf{x}), f_3(\mathbf{x})\big) = \big(\text{ECE},\ \text{NLL},\ -\text{ACC}\big) \tag{43}$$

with inequality constraints

$$c_j(\mathbf{x}) \le 0, \quad j = 1, \dots, J. \tag{44}$$

$$\begin{aligned} y_m(\mathbf{x}) &= f_m(\mathbf{x}) + \varepsilon_m, \quad \varepsilon_m \sim \mathcal{N}(0, \sigma_m^2), \\ \tilde{y}_j(\mathbf{x}) &= c_j(\mathbf{x}) + \tilde{\varepsilon}_j,\ \tilde{\varepsilon}_j \sim \mathcal{N}(0, \tilde{\sigma}_j^2) \end{aligned} \tag{45}$$

For each objective/constraint:

$$f_m \sim \mathcal{GP}\big(\mu_m, k_m\big), \qquad c_j \sim \mathcal{GP}\big(\mu_j^{(c)}, k_j^{(c)}\big) \tag{46}$$

Given data $\mathcal{D}$ and a candidate batch $X = \{\mathbf{x}^{(q)}\}_{q=1}^{Q}$,

$$\mathbf{f}_m(X) \mid \mathcal{D} \sim \mathcal{N}\big(\boldsymbol{\mu}_{m|n}(X),\ \boldsymbol{\Sigma}_{m|n}(X)\big), \tag{47}$$

$$\begin{aligned} \boldsymbol{\mu}_{m|n}(X) &= \mu_m(X) + K_m(X, X_{1:n}) \\ &\quad \times \big(K_m + \sigma_m^2 I\big)^{-1}\big(\mathbf{y}_m - \mu_m(X_{1:n})\big) \end{aligned} \tag{48}$$

$$\begin{aligned} \boldsymbol{\Sigma}_{m|n}(X) &= K_m(X, X) - K_m(X, X_{1:n}) \\ &\quad \times \big(K_m + \sigma_m^2 I\big)^{-1} K_m(X_{1:n}, X) \end{aligned} \tag{49}$$

and analogously for constraints.

With independent constraints,

$$\text{PoF}(X) \approx \prod_{q=1}^{Q} \prod_{j=1}^{J} \Phi\left(\frac{-\mu_{j|n}^{(c)}(\mathbf{x}^{(q)})}{\sqrt{\Sigma_{j|n}^{(c)}(\mathbf{x}^{(q)}, \mathbf{x}^{(q)})}} \right) \tag{50}$$

where Φ is the standard normal CDF.

q-NEHVI acquisition (constrained). Let $\mathbf{r}$ be the reference point and $\text{HV}(\cdot; \mathbf{r})$ the dominated hypervolume. Define

$$\alpha_{\text{q-NEHVI}}(X) = \mathbb{E}\Big[\big(\text{HV}(\tilde{\mathcal{P}} \cup \mathbf{F}(X); \mathbf{r}) - \text{HV}(\tilde{\mathcal{P}}; \mathbf{r})\big)_+, \ \mathbb{I}\{\mathbf{C}(X) \le \mathbf{0}\} \ \Big|\ \mathcal{D}\Big] \tag{51}$$

Table 13. MMLU subjects and tasks.

Subject	**Tasks**
Computer Science (CS)	college computer science, computer security, high school computer science, machine learning
Engineering (Eng)	electrical engineering
Law	international law, jurisprudence, professional law
Health	anatomy, clinical knowledge, college medicine, human aging, nutrition, professional medicine, virology

Table 14. Metrics before vs. after constrained Bayesian optimization. "Before" uses Table 1 settings (Bayesian-LoRA, S=4, early stop); "After (BO)" are results under constrained BO

	ACC ↑		**ECE ↓**		**NLL ↓**		**Hyperparams (BO)**	
Dataset	Before	After (BO)	Before	After (BO)	Before	After (BO)	LR (BO)	WD (BO)
WG-S	70.90	**72.94**	4.90	**2.74**	0.79	**0.54**	9.792×10^{-5}	9.339×10^{-2}
ARC-C	68.90	**69.60**	9.20	**6.10**	**0.77**	0.86	4.955×10^{-4}	2.056×10^{-1}
ARC-E	85.91	**88.38**	5.30	**4.97**	**0.38**	0.39	7.393×10^{-4}	2.707×10^{-2}
WG-M	74.30	**76.34**	**3.00**	7.90	0.62	**0.52**	4.280×10^{-4}	1.000×10^{-2}
OBQA	81.60	**82.80**	**5.70**	5.84	**0.49**	0.54	6.552×10^{-5}	9.712×10^{-2}
BoolQ	86.10	**86.41**	**2.10**	3.10	0.29	**0.28**	5.421×10^{-4}	3.582×10^{-1}

where $\tilde{\mathcal{P}}$ is a sample of the posterior Pareto set (feasible and non-dominated), $\mathbf{F}(X)$ stacks the objective draws, and $\mathbf{C}(X)$ the constraint draws.

With Cholesky factors $\mathbf{L}_{m|n}(X)$ of (49),

$$\begin{aligned} \mathbf{F}_m^{(t)}(X) &= \boldsymbol{\mu}_{m|n}(X) + \mathbf{L}_{m|n}(X)\mathbf{z}_m^{(t)}, \\ \mathbf{z}_m^{(t)} &\sim \mathcal{N}(\mathbf{0}, I) \end{aligned} \tag{52}$$

and similarly $\mathbf{C}^{(t)}(X)$. Sampling $\tilde{\mathcal{P}}^{(t)}$ from the posterior of historical designs, the MC estimator is

$$\begin{aligned} \widehat{\alpha}_{\text{q-NEHVI}}(X) = \frac{1}{T}\sum_{t=1}^{T} \Big(&\text{HV}(\tilde{\mathcal{P}}^{(t)} \cup \mathbf{F}^{(t)}(X); \mathbf{r}) \\ &- \text{HV}(\tilde{\mathcal{P}}^{(t)}; \mathbf{r})\Big)_+ \mathbb{I}\{\mathbf{C}^{(t)}(X) \le \mathbf{0}\} \end{aligned} \tag{53}$$

A PoF-weighted variant:

$$\begin{aligned} \widehat{\alpha}^{(\text{PoF})}_{\text{q-NEHVI}}(X) = \Bigg[&\frac{1}{T}\sum_{t=1}^{T} \Big(\text{HV}\big(\tilde{\mathcal{P}}^{(t)} \cup \mathbf{F}^{(t)}(X); \mathbf{r}\big) \\ &- \text{HV}\big(\tilde{\mathcal{P}}^{(t)}; \mathbf{r}\big)\Big)_+\Bigg] \\ &\times \text{PoF}(X) \end{aligned} \tag{54}$$

The reference point is then obtained as follows:

$$\mathbf{r} = (r_1, r_2, r_3), \qquad r_m < \min_{\text{historical feasible}} f_m \tag{55}$$

Table 16. Trainable parameters corresponding to different ranks.

Rank	4	9	16	32	64	128
Trainable Parameters	3.4M	4.9M	7.0M	12M	22M	43M

Table 15. Full Pareto candidates after constrained BO (all datasets combined).

Dataset	Cand.	Acc	NLL	ECE (%)	LR	Weight Decay
WinoGrande-M	1	0.7753	0.6010	12.29	6.319×10^{-4}	4.635×10^{-1}
WinoGrande-M	2	0.7682	0.5499	8.34	6.904×10^{-4}	1.000×10^{-2}
WinoGrande-M	3	0.7642	0.5329	8.15	6.319×10^{-4}	4.635×10^{-1}
WinoGrande-M	4	0.7634	0.5260	7.92	4.280×10^{-4}	1.000×10^{-2}
WinoGrande-M	5	0.7342	0.5354	3.71	3.434×10^{-4}	1.152×10^{-2}
WinoGrande-S	1	0.7445	0.6062	12.06	9.792×10^{-5}	9.339×10^{-2}
WinoGrande-S	2	0.7342	0.5870	9.90	7.832×10^{-5}	8.0912×10^{-2}
WinoGrande-S	3	0.7294	0.5474	2.74	9.792×10^{-5}	9.339×10^{-2}
WinoGrande-S	4	0.7033	0.5740	2.42	8.810×10^{-5}	7.783×10^{-2}
ARC-C	1	0.6959	0.8616	6.10	4.955×10^{-5}	2.056×10^{-1}
ARC-C	2	0.6858	0.8556	9.16	9.581×10^{-5}	5.000×10^{-1}
ARC-C	3	0.6014	1.0015	4.41	6.081×10^{-5}	3.141×10^{-1}
ARC-E	1	0.8891	0.4184	5.89	4.921×10^{-4}	5.000×10^{-1}
ARC-E	2	0.8838	0.3996	4.97	5.456×10^{-4}	2.707×10^{-2}
ARC-E	3	0.8680	0.4495	4.39	7.394×10^{-4}	5.000×10^{-1}
ARC-E	4	0.8556	0.4797	2.99	6.319×10^{-4}	4.635×10^{-1}
OBQA	1	0.8380	0.5499	7.18	1.180×10^{-3}	4.635×10^{-1}
OBQA	2	0.8280	0.5411	5.84	2.135×10^{-4}	2.025×10^{-1}
BoolQ	1	0.8641	0.2841	3.10	5.421×10^{-4}	3.582×10^{-1}
BoolQ	2	0.8572	0.3025	2.54	4.987×10^{-4}	2.041×10^{-1}
BoolQ	3	0.8490	0.3152	1.86	6.103×10^{-4}	5.000×10^{-1}

Table 17. Training hyperparameters for deterministic LoRA and Bayesian LoRA fine-tuning on the MATH dataset. Bayesian LoRA uses a structured Bayesian low-rank update with rank $r_{\text{bayes}} = 9$.

Category	Hyperparameter (Value)
Model & Tokenizer	
Model name	`Qwen/Qwen3-30B-A3B-Instruct-2507`
Max input length	1024
BF16 precision	True
FP16 precision	False
Load in 8-bit	False
Gradient checkpointing	Optional (default: disabled)
Pad token	EOS token
Training	
Epochs	1
Batch size (per device)	1
Gradient accumulation steps	1
Learning rate	1e–5
Warmup steps	100
Optimizer	AdamW (PyTorch)
Max grad norm	1.0
Logging steps	50
Save strategy	Per epoch
Save total limit	4
Disable tqdm	False
Dataset & Prompt	
Dataset	`DigitalLearningGmbH/MATH-lighteval`
Split	train
Prompt type	qwen25-math-cot
Apply chat template	True
Max tokens per call	1534
Deterministic LoRA	
LoRA rank r	8
LoRA α	16
LoRA dropout	0.05
Target modules	q_proj, v_proj
Bias	none
Task type	Causal LM
Bayesian LoRA (ours)	
Bayesian LoRA layer	`BayesianLoRALayer`
Bayesian rank (r_{bayes})	9
Linear layer override	`nn.Linear` $\rightarrow$ `BayesianLoRALayer`
Posterior treatment	Variational / structured low-rank
Uncertainty sampling	Enabled (Bayesian forward samples)
Merge for inference	supported via `merge_and_unload`
Evaluation (unused in this script)	
Temperature	0
Top-p	1.0
Num sampling	1
vLLM eval	Optional
Eval batch size	1